

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING IN POLYMER INFORMATICS

Day 2

Supervised Learning and Machine Learning Models

Code Link

- <https://github.com/ytl0410/Polymer-Informatics>
- https://drive.google.com/drive/folders/1blO2hW-e2XyUtUVUnKVJMNgO9eleLPDu?usp=drive_link
- https://uwprod-my.sharepoint.com/:u:/g/personal/tyue4_wisc_edu/EYDJnNn5xK9GjfSJYCcbqwUBn7Xm_VLcK_rXSIhrt-MHGQ?e=6gMDsX



Python Environment:

- `conda create -n py39 python=3.9`
- `conda install -c conda-forge cudatoolkit=11.2 cudnn=8.1.0`
- `python -m pip install "tensorflow<2.11"`
- `pip install numpy==1.26.4`
- `pip install matplotlib==3.8.0`
- `pip install pandas==1.5.3`
- `pip install rdkit==2023.9.1`
- `pip install scikit-learn==1.3.0`
- `pip install keras-tuner==1.4.5`
- `pip install seaborn==0.13.0`



Open-Source Cheminformatics
and Machine Learning

<https://www.rdkit.org/docs/Install.html>

With pip (in Google Colab):

- `pip install rdkit`



TensorFlow

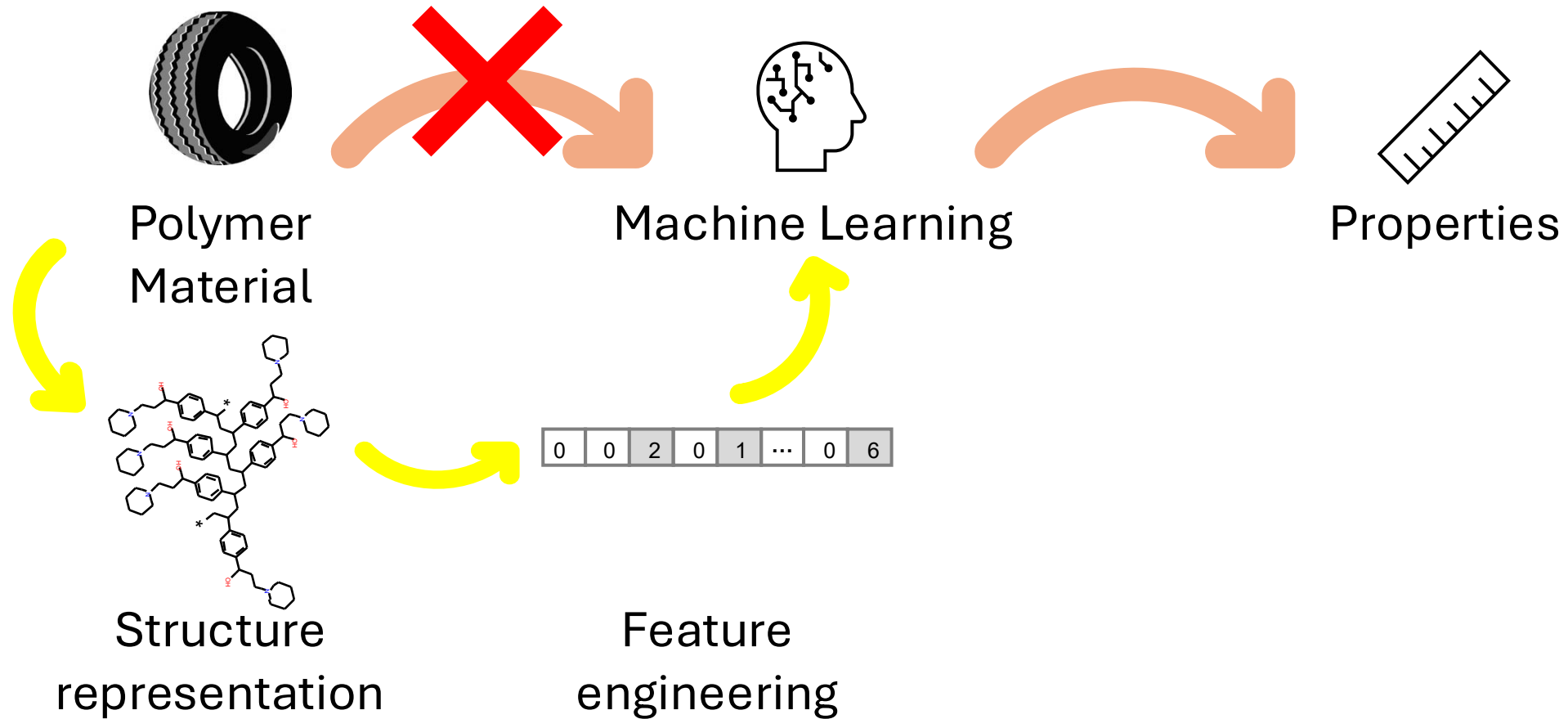
OR



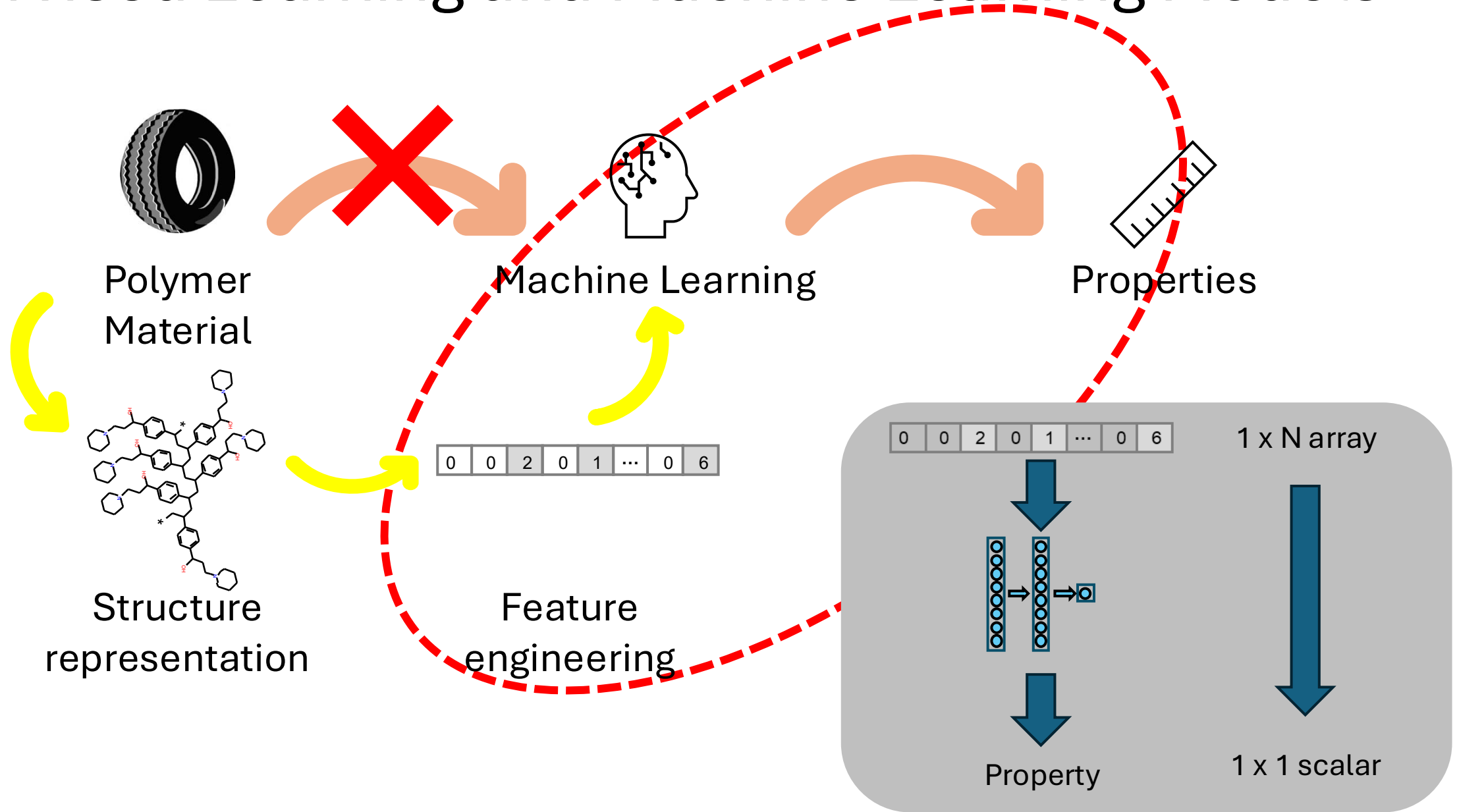
<https://www.tensorflow.org/install>

Version	Python version	Compiler	Build tools	cuDNN	CUDA
tensorflow-2.7.0	3.7-3.9	GCC 7.3.1	Bazel 3.7.2	8.1	11.2
tensorflow-2.6.0	3.6-3.9	GCC 7.3.1	Bazel 3.7.2	8.1	11.2
tensorflow-2.5.0	3.6-3.9	GCC 7.3.1	Bazel 3.7.2	8.1	11.2
tensorflow-2.4.0	3.6-3.8	GCC 7.3.1	Bazel 3.1.0	8.0	11.0
tensorflow-2.3.0	3.5-3.8	GCC 7.3.1	Bazel 3.1.0	7.6	10.1
tensorflow-2.2.0	3.5-3.8	GCC 7.3.1	Bazel 2.0.0	7.6	10.1

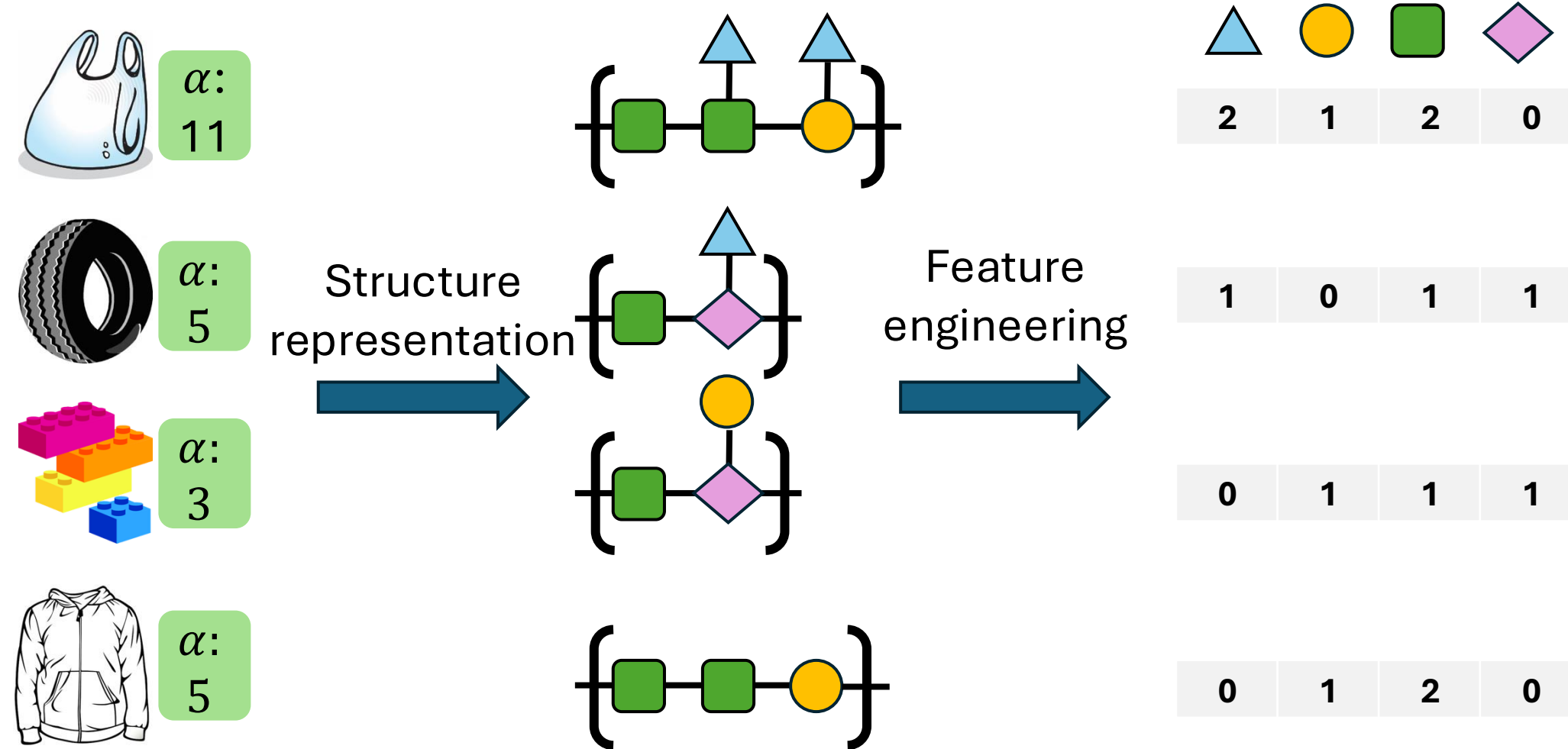
Supervised Learning and Machine Learning Models



Supervised Learning and Machine Learning Models

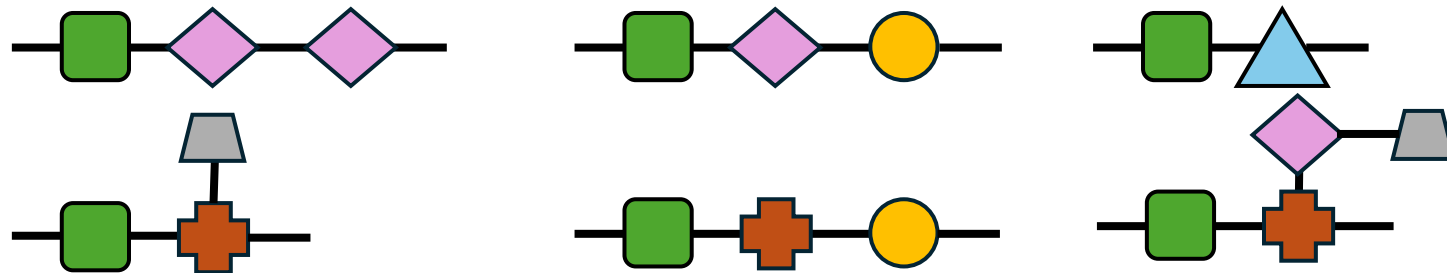
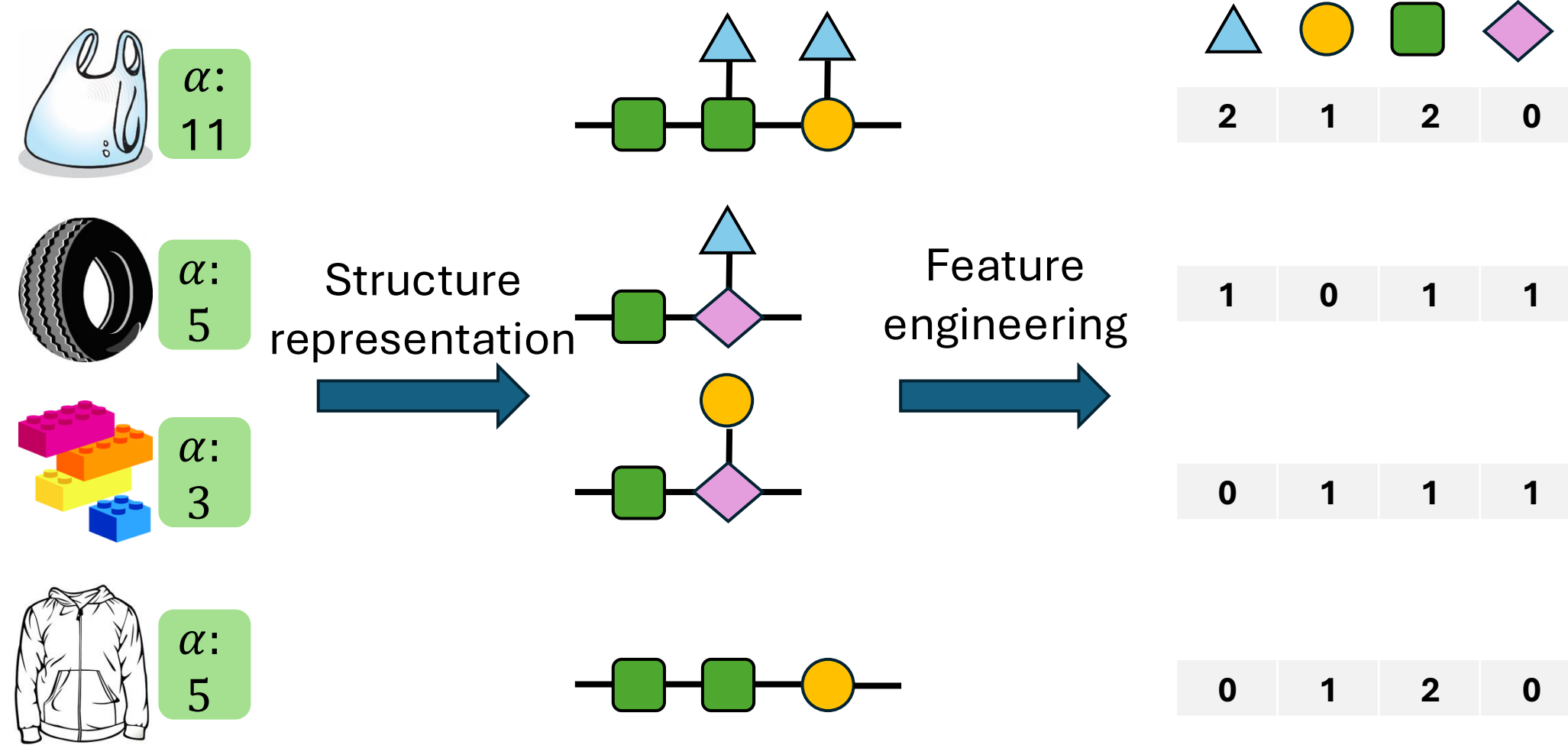


Supervised Learning and Machine Learning Models

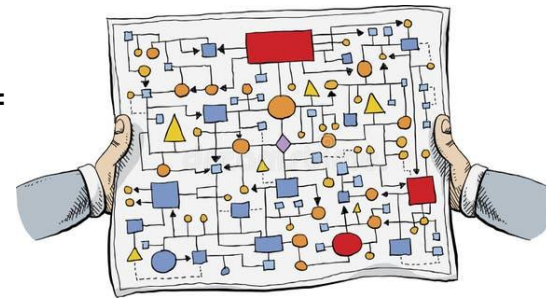


1 x 4 array $\xrightarrow{3 \times \text{blue triangle} + 1 \times \text{yellow circle} + 2 \times \text{green square} + 0 \times \text{purple diamond}}$ 1 x 1 scalar

Supervised Learning and Machine Learning Models



Large
Amount of
Samples
...



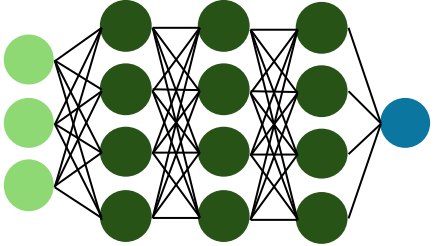
Complex
Nonlinear
Mapping

Prediction Models Are Typically Used for Candidate Library Screening



What's the T_g of this polymer? It's too difficult to characterize.

C/C=C/C(c1ccccc1)(c2ccccc2)C



Prediction Models

Input:

1	0	1	0	0	...	0	0	1	0	1	0
---	---	---	---	---	-----	---	---	---	---	---	---

Output:

367°C



We have discovered a material that outperforms the state of the art!

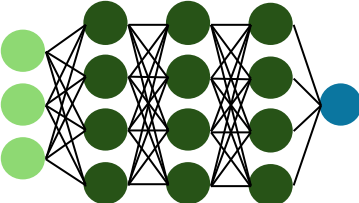




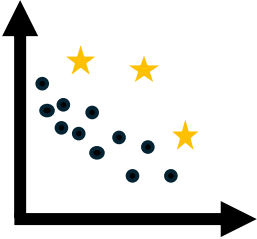
I have 2,000 candidate polymer structures—which ones should I prioritize for testing first?



I can use prediction models to identify promising candidates. These structures show higher predicted performance, indicating strong potential.



Prediction Models





I will begin with the most promising candidates.

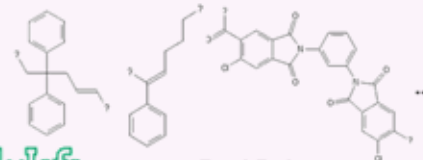


1. Candidate libraries



Real Polymer Structures

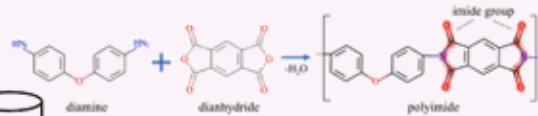
PolyInfo ~18,000 Real Polymers





Hypothetical Polymer Structures

~8 million hypothetical polyimide

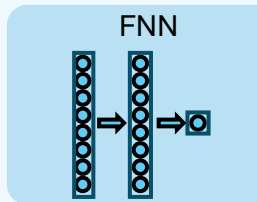


3. High throughput screening

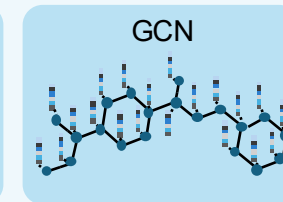
A 3D scatter plot showing the relationship between glass transition temperature (T_g in °C), a parameter k , and band gap energy (E_g in eV). The T_g axis ranges from 100.0 to 350.0. The k axis ranges from 2.25 to 4.75. The E_g axis ranges from 3.00 to 4.50. Data points are colored by T_g (red for high, blue for low). Several points are highlighted with yellow stars.

A diagram illustrating the high-throughput screening process. It starts with a large library of polymer candidates (represented by purple and yellow dots) being screened. A small portion of the library is selected and synthesized into a polymer film. The synthesis reaction is shown: Nc1ccc(N)cc1 + O=C1OC(=O)c2cc3C(=O)N(C3=O)c21 in p -Cresol or p -Chlorophenol, refluxed overnight to form a polyimide. The resulting polymer is then characterized using a thermometer and a ruler.

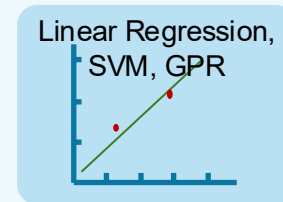
2. ML-based prediction models



FNN

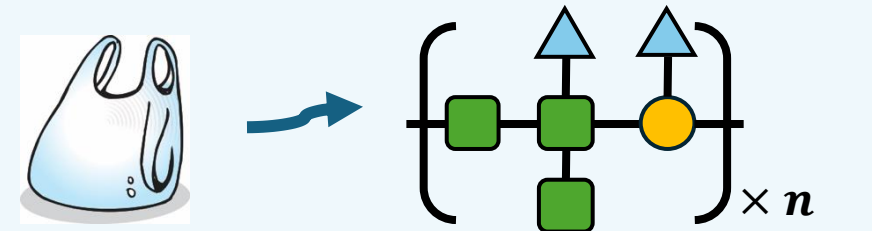


GCN

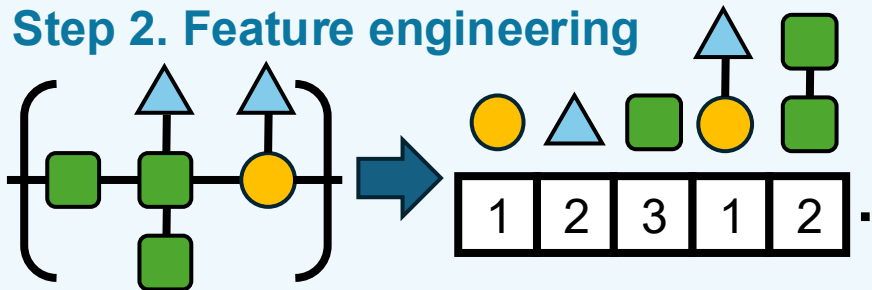


Linear Regression, SVM, GPR

Step 1. Structure representation



Step 2. Feature engineering



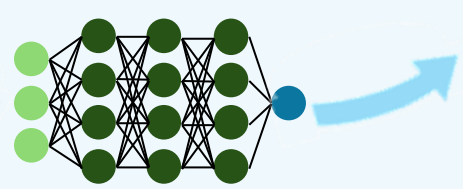
Step 3. ML algorithms

1

2

3

...


$$t = f(wa + b)$$

T_g

E_g

k

Links your Google drive to the Colab session

```
[ ] from google.colab import drive
drive.mount('/content/drive')
```

Permit this notebook to access your Google Drive files?

This notebook is requesting access to your Google Drive files. Granting access to Google Drive will permit code executed in the notebook to modify files in your Google Drive. Make sure to review notebook code prior to allowing this access.

No thanks **Connect to Google Drive**

Sign in with Google

You're signing back in to Google Drive for desktop

Review Google Drive for desktop's [Privacy Policy](#) and [Terms of Service](#) to understand how Google Drive for desktop will process and protect your data.

To make changes at any time, go to your [Google Account](#).

Learn more about [Sign in with Google](#).

tyue4@wisc.edu

Cancel **Continue**

Make sure you trust Google Drive for desktop

Review Google Drive for desktop's [Privacy Policy](#) and [Terms of Service](#) to understand how Google Drive for desktop will process and protect your data.

To make changes at any time, go to your [Google Account](#).

Learn how Google helps you [share data safely](#).

Cancel **Continue**

Runtime duration

1m

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Output

Links your Google drive to the Colab session

✓
0s

[2] path = `'/content/drive/MyDrive/SPE/'`  **Modify the path to the directory where your files are stored**

Check the path name is assigned correctly.

✓
0s

[3] `!echo $path`
`import os`
`os.path.isdir(path)`

 `/content/drive/MyDrive/SPE/`
`True`

 **The path is set correctly**

✓
0s

 `# This is needed so can import packages from a different path than standard libraries`
`import sys`
`sys.path.append(path)`

Check the Python packages that are available by default in the Colab environment.

Set up Python environment

Pre-installed packages 

 `!pip list`

Package	Version

absl-py	1.4.0
accelerate	0.32.1
aiohappyeyeballs	2.4.0
aiohttp	3.10.5
aiosignal	1.3.1
alabaster	0.7.16
albucore	0.0.13
albumations	1.4.14
altair	4.2.2
annotated-types	0.7.0
anyio	3.7.1
argon2-cffi	23.1.0
argon2-cffi-bindings	21.2.0
array_record	0.5.1
arviz	0.18.0
asn1crypto	1.5.1
astropy	6.1.2
astropy-iers-data	0.2024.8.26.0.31.57

Need to install some python packages which is not present on Colab environment by default.

✓
11s

 `!pip install rdkit`

 Collecting rdkit
 Downloading rdkit-2025.3.5-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (4.1 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from rdkit) (2.0.2)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from rdkit) (11.3.0)
 Downloading rdkit-2025.3.5-cp312-cp312-manylinux_2_28_x86_64.whl (36.2 MB)
----- 36.2/36.2 MB 51.3 MB/s eta 0:00:00

Installing collected packages: rdkit
Successfully installed rdkit-2025.3.5

 **Install rdkit package successfully**

Import all required packages

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

import rdkit
from rdkit import Chem
from rdkit.Chem import AllChem
from rdkit.Chem import Descriptors
from rdkit.Chem import rdMolDescriptors
from rdkit.Chem.Draw import IPythonConsole
from rdkit.Chem import Draw

from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.svm import SVR
from sklearn.linear_model import Lasso, LassoCV
from sklearn.model_selection import train_test_split, cross_val_predict, cross_validate
from sklearn.preprocessing import StandardScaler
from sklearn.utils import shuffle
from math import sqrt

from tensorflow.keras.models import Sequential, save_model, load_model
from tensorflow.keras.layers import Dense, Activation, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow import keras
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Conv1D, MaxPooling1D, Dense, Flatten, Activation
import time
from sklearn.metrics import r2_score

from collections import Counter
import pickle
import pandas as pd
import collections
import seaborn as sns

from rdkit import rdBase
rdBase.DisableLog('rdApp.warning')
```

Scientific computing package

Plotting

Data management

Cheminformatics toolkit

scikit-learn
Library for classical machine learning algorithms

Deep learning framework

Importing the Dataset

```
[8] # Read in the Tg data  
Tg_df = pd.read_csv(os.path.join(path, "Tg.csv"))
```

→ Dataset file name

```
# View the dataset (dataframe)  
Tg_df
```

	Smiles	Tg(C)
0	*C*	-63.48
1	*C(C*)C	-2.73
2	*C(C*)CC	-22.54
3	*C(C*)CCC	-32.29
4	*C(C*)C(C)C	10.97
...	...	...
6901	*C1C(C(C(C1)(O)C(F)(F)F)(F)F)(C(C*)(F)F)F	152.00
6902	*C1C(CC(C1)CC(O)(C(F)(F)F)C(F)(F)F)(C(C*)(F)F)F	98.00
6903	*C1C(CC(C1)C(O)(C(F)(F)F)C(F)(F)F)(C(C*)(F)F)F	118.00
6904	*c1cc2C3(CCC(CC3)(C3=NC(CO3)c3ccccc3)C3=NC(CO3...	161.00
6905	*c1cc2C3(CCC(CC3)(C3=NC(CO3)Cc3ccccc3)C3=NC(CO...	142.00

6906 rows x 2 columns



→ Interactive table



→ Chart



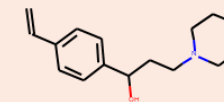
↓
Dataset size

↓
First column:
Smiles of repeat units

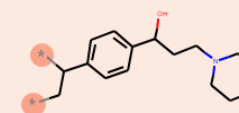
↓
Second column: label
values / attribute values

Structure

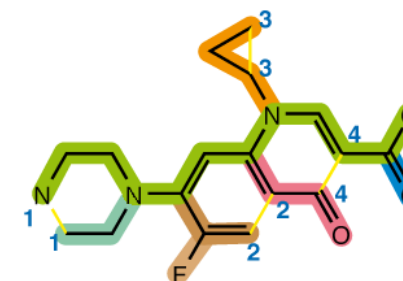
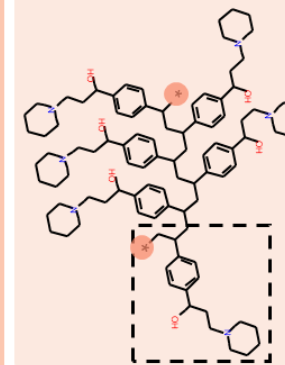
Monomer



Repeat unit



Repeat units polymerized



N1CCN(CC1)C(C(F)=C2)=CC(=C2C4=O)N(C3CC3)C=C4C(=O)O

Morgan fingerprint

Search radius (Generally set to 3)



The length of the fingerprint bit vector
(Generally set to 1024 or 2048)

```
# Generation of Morgan fingerprint

fn = Tg_df.Smiles.apply(Chem.MolFromSmiles).apply(lambda m: AllChem.GetMorganFingerprintAsBitVect(m, radius=3, nBits=1024))

# Convert the Morgan fingerprints into a DataFrame with 1024 columns.
MF = pd.DataFrame()
columns = []
for i in range(1024):
    col_name = f'Bit_{i+1}'
    new_column = fn.apply(lambda x: int(bool(x.GetBit(i))))
    columns.append(pd.Series(new_column, name=col_name))
MF = pd.concat([MF] + columns, axis=1)
MF
```

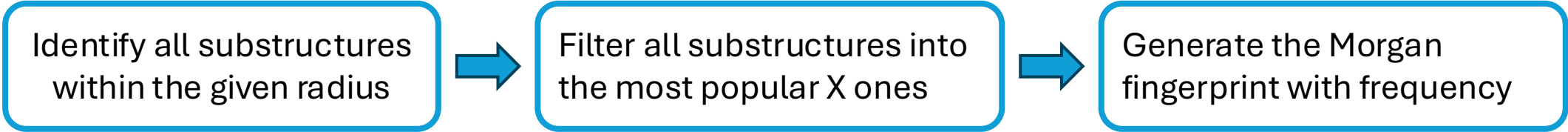
	Bit_1	Bit_2	Bit_3	Bit_4	Bit_5	Bit_6	Bit_7	Bit_8	Bit_9	Bit_10	...	Bit_1015	Bit_1016	Bit_1017	Bit_1018	Bit_1019	Bit_1020	Bit_1021	Bit_1022	Bit_1023	Bit_1024
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	1	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	0	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	1	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6901	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0
6902	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0
6903	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0
6904	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0
6905	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	1	0	0	0	0

6906 rows x 1024 columns



The generated Morgan fingerprint

Morgan fingerprint with frequency



```
# Generate the Morgan fingerprint with frequency after determining the number of dimensions

Columns = Zero_Sum[Zero_Sum < NumberOfZero].index
Substructure_list = list(polymer.keys())
MFF = MFF[Columns]
MFF
```

	15	36	127	142	169	205	210	240	243	259	...	26115	26117	26127	26137	26154	26162	26199	26215	26231	26286
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6901	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6902	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6903	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6904	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
6905	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0



The generated Morgan fingerprint with frequency

Descriptors



```
[33] # Apply Min-Max scaling to normalize the descriptors data
# MinMaxScaler scales the features to a range between 0 and 1
minmax_descriptors = MinMaxScaler().fit_transform(descriptors_df_lowcorr)
# Convert the normalized descriptors back into a DataFrame
# The column names are preserved from the original DataFrame
minmax_descriptors_df = pd.DataFrame(minmax_descriptors,columns=descriptors_df_lowcorr.columns)
```

Apply Min-Max scaling to normalize the descriptors data

	MaxAbsEStateIndex	MinAbsEStateIndex	MinEStateIndex	qed	SPS	MolWt	NumRadicalElectrons	FpDensityMorgan1	AvgIpc	BalabanJ	...	fr_quatN	fr_sulfide	fr_sulfonamd	fr_sulfone	fr_term_acetylene	fr_tetrazole	f
0	0.004263	0.255605	1.000000	0.386171	0.159898	0.000000	0.0	0.919831	0.061115	0.168029	...	0.0	0.0	0.0	0.0	0.0	0.0	
1	0.045281	0.219090	0.988061	0.452004	0.246651	0.012819	0.0	0.679325	0.182239	0.298480	...	0.0	0.0	0.0	0.0	0.0	0.0	
2	0.041279	0.217061	0.987397	0.492744	0.194344	0.019229	0.0	0.559072	0.253584	0.329368	...	0.0	0.0	0.0	0.0	0.0	0.0	
3	0.039829	0.214505	0.986562	0.520777	0.162960	0.025638	0.0	0.486920	0.261423	0.340539	...	0.0	0.0	0.0	0.0	0.0	0.0	
4	0.043925	0.204213	0.983197	0.508399	0.181331	0.025638	0.0	0.438819	0.252450	0.385492	...	0.0	0.0	0.0	0.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
6901	0.796884	0.325828	0.357296	0.840723	0.533627	0.117020	0.0	0.254902	0.445186	0.434493	...	0.0	0.0	0.0	0.0	0.0	0.0	
6902	0.821072	0.133725	0.342801	0.838366	0.371313	0.144469	0.0	0.221218	0.510769	0.375612	...	0.0	0.0	0.0	0.0	0.0	0.0	
6903	0.817053	0.228554	0.333116	0.861113	0.388008	0.138059	0.0	0.222363	0.512045	0.407853	...	0.0	0.0	0.0	0.0	0.0	0.0	
6904	0.322520	0.007616	0.867324	0.098678	0.215954	0.384359	0.0	0.072509	0.917571	0.075409	...	0.0	0.0	0.0	0.0	0.0	0.0	
6905	0.324449	0.008864	0.870339	0.086545	0.210625	0.397178	0.0	0.072675	0.915227	0.072584	...	0.0	0.0	0.0	0.0	0.0	0.0	

6906 rows x 144 columns

The generated descriptors

Establishing training/test split

▼ Morgan fingerprint as feature engineering

```
[ ] X = MF  
    y = Tg_df['Tg(C)']
```

→ The generated Morgan fingerprint

→ Column name

Then, we use the `train_test_split()` method from the `scikit-learn` package to generate the split. In this case, our input data `X` and output data `y` are split into 4 parts:

- `X_train`: training set input data
- `X_test`: test set input data
- `y_train`: training set output data
- `y_test`: test set output data

```
[ ] # Generate train/test split by reserving 10% of data as test set
```

```
test_fraction = 0.1  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_fraction, shuffle=True, random_state = 11)
```

→ Reserving 10% of data as test set (Typically 20%, when sufficient data is available)

↓

Random seed
Consistently get the same training and testing datasets

Lasso regularization

```
[ ] from sklearn.linear_model import Lasso

# Create a Lasso model instance with a specified alpha parameter
# Alpha controls the strength of the regularization
# A higher alpha means stronger regularization and a simpler model
alpha_value = 1
lasso = Lasso(alpha=alpha_value)

# Train the Lasso model using the training data
lasso.fit(X_train, y_train)
```

Import from scikit-learn

Hyperparameter alpha

Fit the model

Lasso

Lasso(alpha=1)

```
[ ] # Calculate and print the R^2 score for the training set
y_pred_train = lasso.predict((X_train))
print("Train set R^2: ", r2_score(y_train, y_pred_train))
# Calculate and print the R^2 score for the test set
y_pred_test = lasso.predict((X_test))
print("Test set R^2: ", r2_score(y_test, y_pred_test))
```

Predict (on training set)

Calculate R^2

Predict (on test set)

Calculate R^2

Train set R^2: 0.7346571619684173

Test set R^2: 0.7230192626650318

Output training & test R^2

Lasso (least absolute shrinkage and selection operator)

Input

$$x = (x_1, x_2, \dots, x_p)$$

Output

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_px_p$$

Objective function

$$\min \left[\frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \sum_{j=0}^p |w_j| \right]$$

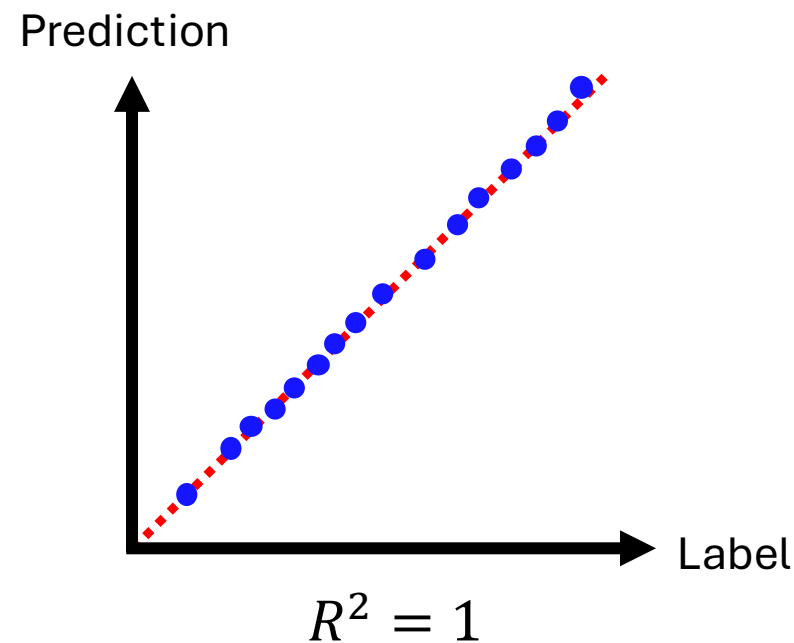
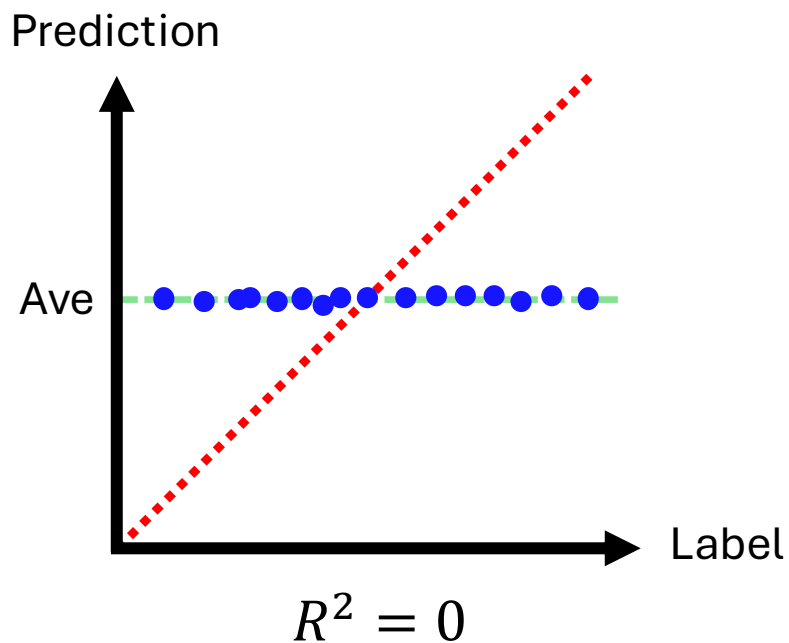
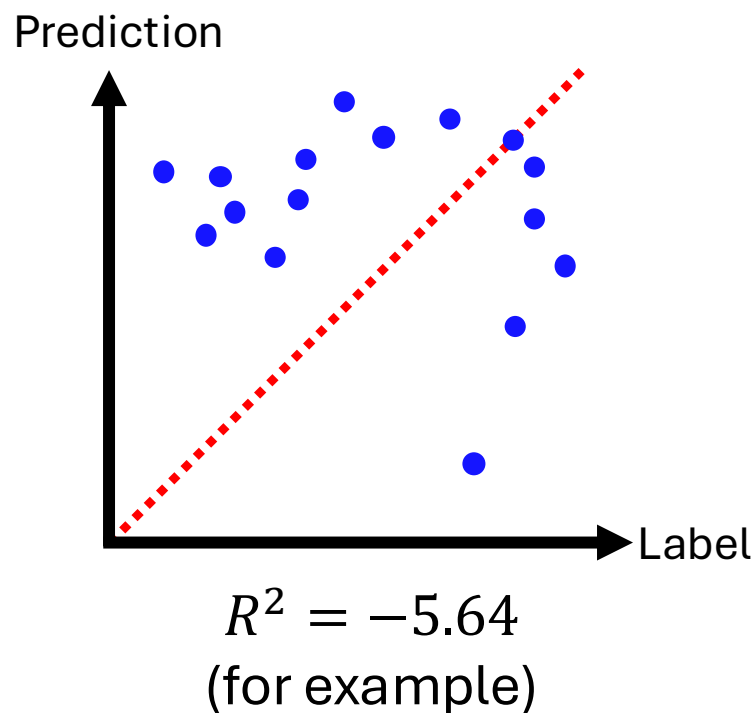
Linear Regression

L1 penalty term

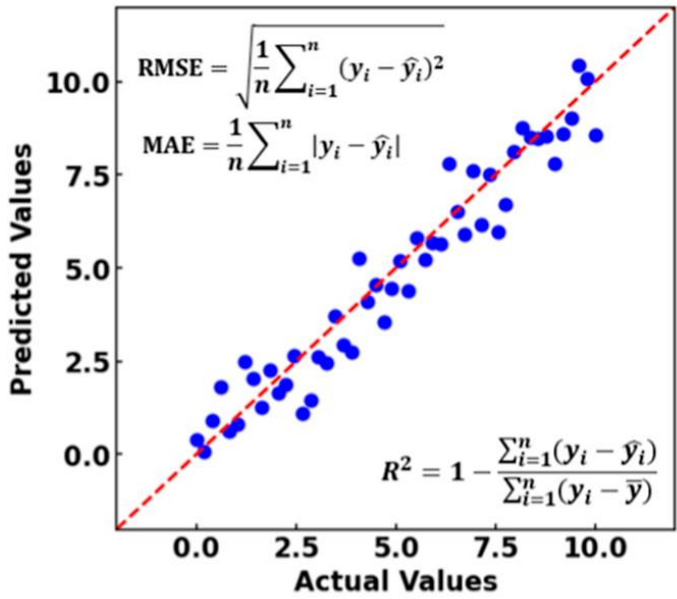
Coefficient of determination (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^n ((y_i - \hat{y}_i)^2)}{\sum_{i=1}^n ((y_i - \bar{y})^2)}$$

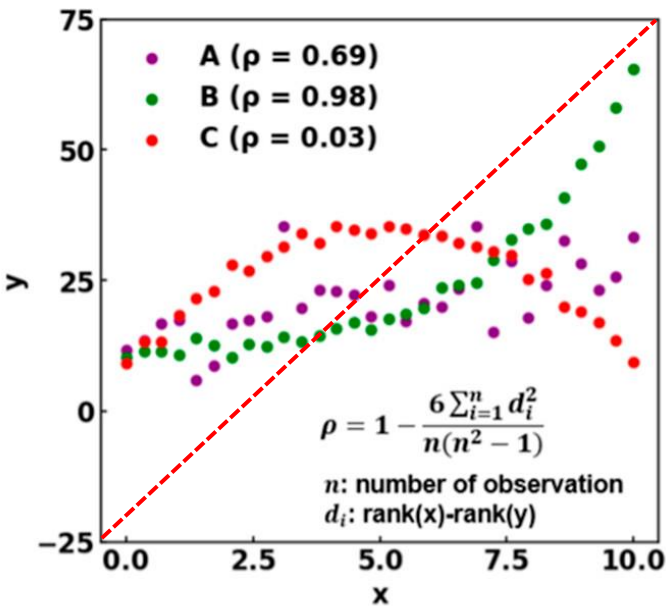
How much error is reduced compared to a mean-value baseline.



Evaluating Model Performance and Reliability



(a)



(b)

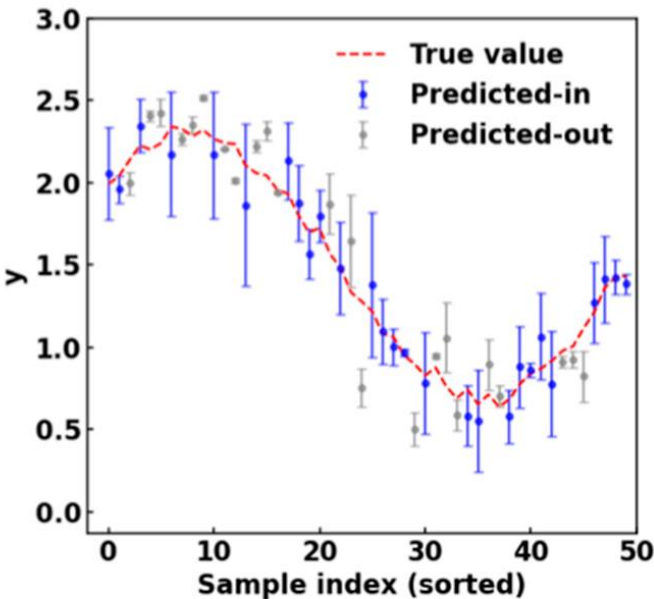
(a)

Root Mean Squared Error (RMSE)
Mean Absolute Error (MAE)

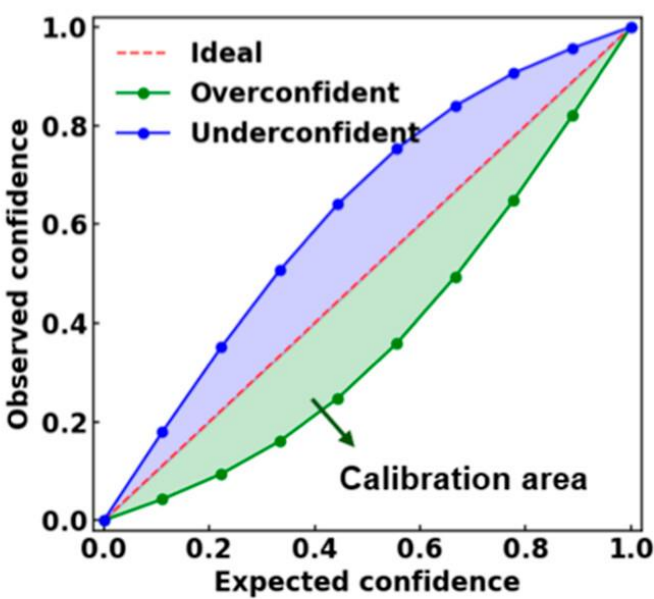
(b)

Spearman's Rank Correlation Coefficient (ρ)

- ρ values close to 1 indicate a strong monotonic relationship between ranks of x and y



(c)



(d)

(c)

Confidence interval assessment for calibration

- The proportion of predictions where true values fall within the predicted confidence intervals

(d)

Calibration curve and area
Observed vs expected confidence from 0 to 1

Lasso regularization

```
plt.figure(figsize=(5,5))

ax=plt.subplot(1, 1, 1)
# Plot the predicted vs. actual values for the training set in blue
plt.plot(y_train, y_pred_train, 'b.')
# Plot the predicted vs. actual values for the test set in green
plt.plot(y_test, y_pred_test, 'g.')
# Set the labels for the x-axis and y-axis
plt.xlabel("Experiment (°C)", fontsize=12)
plt.ylabel("Prediction (°C)", fontsize=12)
# Determine the range for the x-axis and y-axis based on the training data
x0, x1 = min(y_train), max(y_train)
length = 750
x_start, x_end = x0*0.9, x1*1.1
# Set the font size and weight for the x and y ticks
plt.xticks(fontsize=12, fontweight='normal')
plt.yticks(fontsize=12, fontweight='normal')
plt.title("Tg", fontsize=12, fontweight='normal')
plt.gca().set_aspect("equal", adjustable="box")
# Plot the unit line (a line with a slope of 1) in red
plt.plot(np.arange(x_start, x_end, (x_end - x_start) / 10),
np.arange(x_start, x_end, (x_end - x_start) / 10), 'r-')
# Add text annotations to display the R^2 scores for the training and test sets
plt.text(x_start + (x_end - x_start)*0, x_end - (x_end - x_start)*0.12, "Train $R^2={:.2f}$".format(r2_score(y_train, y_pred_train)),{'color':"b"}, fontsize=14)
plt.text(x_start + (x_end - x_start)*0.42, x_end - (x_end - x_start)*0.12, "Test $R^2={:.2f}$".format(r2_score(y_test, y_pred_test)),{'color':"g"}, fontsize=14)
```

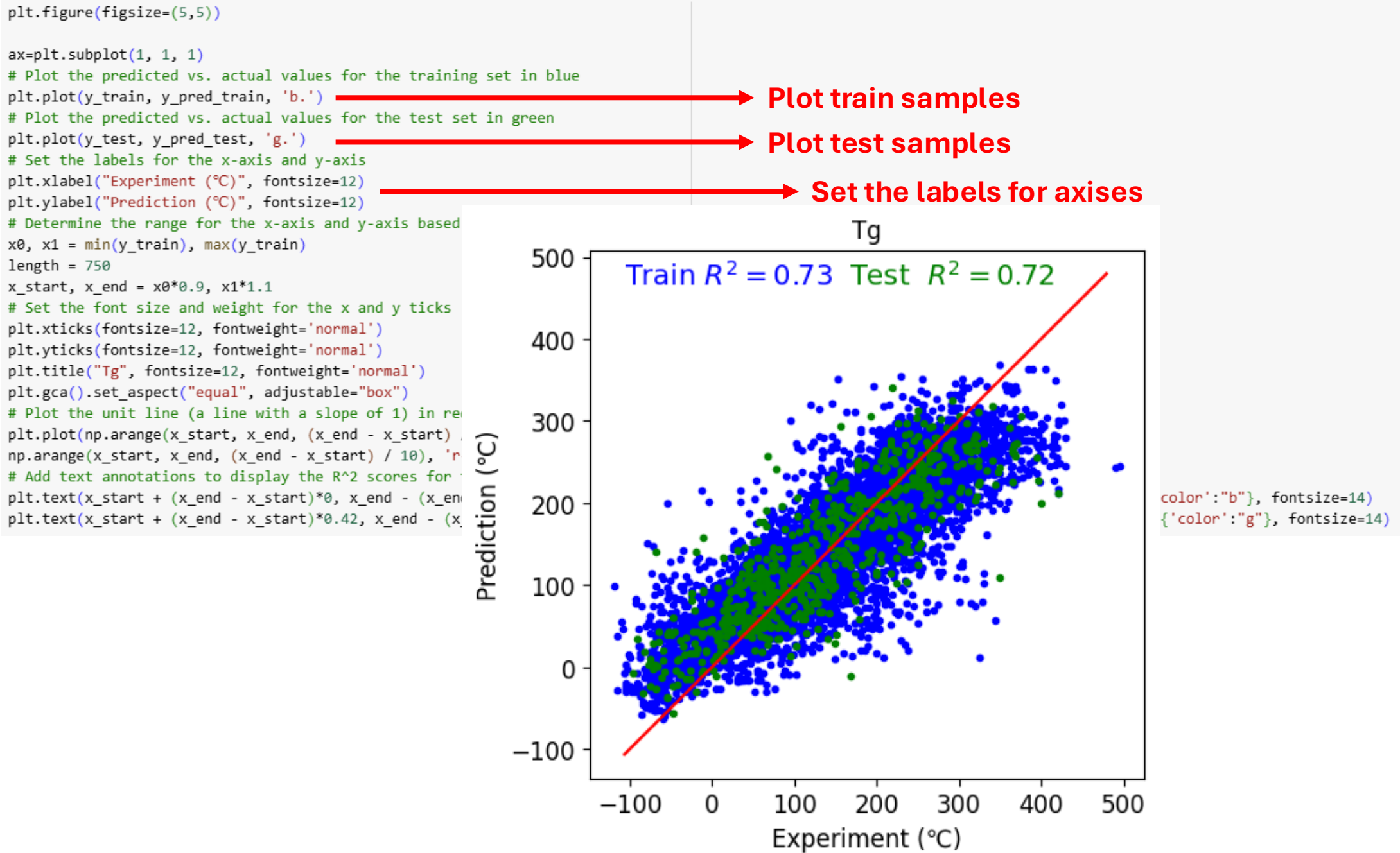
Plot train samples

Plot test samples

Set the labels for axes

Plot the unit line

Lasso regularization



Lasso regularization

```
# Define the range of alpha values
alphas = np.logspace(-4, 1, 10) # 10 points from 10^-5 to 10

# Initialize lists to store R^2 scores
r2_train_scores = []
r2_test_scores = []
```

Set the range of alpha values from 10^{-5} to 10

```
# Iterate over alpha values
```

```
for alpha in alphas:
```

```
    lasso = Lasso(alpha=alpha)
```

```
    lasso.fit(X_train, y_train)
```

Train the Lasso model for each alpha value

```
    # Predict and compute R^2 scores for training and test sets
```

```
    y_pred_train = lasso.predict(X_train)
```

```
    y_pred_test = lasso.predict(X_test)
```

Predict

```
    r2_train = r2_score(y_train, y_pred_train)
```

```
    r2_test = r2_score(y_test, y_pred_test)
```

Calculate R^2

```
    r2_train_scores.append(r2_train)
```

```
    r2_test_scores.append(r2_test)
```

Record R^2

Lasso regularization

```
# Define the range of alpha values
alphas = np.logspace(-4, 1, 10) # 10 points from 10^-5 to 10

# Initialize lists to store R^2 scores
r2_train_scores = []
r2_test_scores = []
```

Set the range of alpha values from 10^{-5} to 10

```
# Iterate over alpha values
```

```
for alpha in alphas:
```

```
    lasso = Lasso(alpha=alpha)
```

```
    lasso.fit(X_train, y_train)
```

Train the Lasso model for each alpha value

```
    # Predict and compute R^2 scores for training and test sets
```

```
    y_pred_train = lasso.predict(X_train)
```

```
    y_pred_test = lasso.predict(X_test)
```

Predict

```
    r2_train = r2_score(y_train, y_pred_train)
```

```
    r2_test = r2_score(y_test, y_pred_test)
```

```
    r2_train_scores.append(r2_train)
```

```
    r2_test_scores.append(r2_test)
```

Plot R^2 vs α

```
# Plotting the R^2 scores
```

```
plt.figure(figsize=(10, 6))
```

```
# Plot R^2 scores for both training and testing sets
```

```
plt.plot(alphas, r2_train_scores, marker='o', color='blue', label='Training Set R^2')
```

```
plt.plot(alphas, r2_test_scores, marker='o', color='orange', label='Testing Set R^2')
```

```
plt.xscale('log')
```

```
plt.xlabel('Alpha')
```

```
plt.ylabel('R^2 Score')
```

```
plt.title('Training and Testing R^2 Scores vs Alpha')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



Random forests

Random seed for initialization of parameters

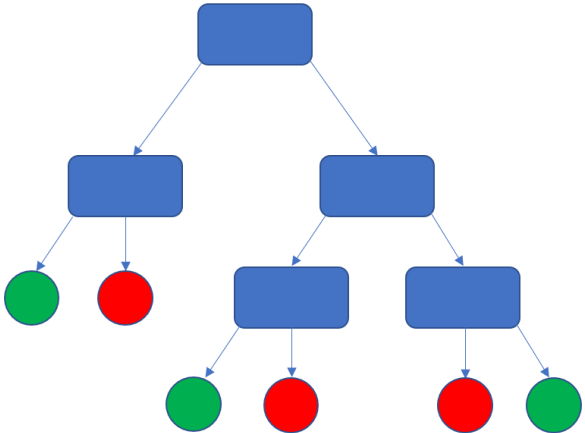
```
from sklearn.ensemble import RandomForestRegressor

# Create a RandomForestRegressor model instance
rf_regressor = RandomForestRegressor(n_estimators=100, random_state=42)

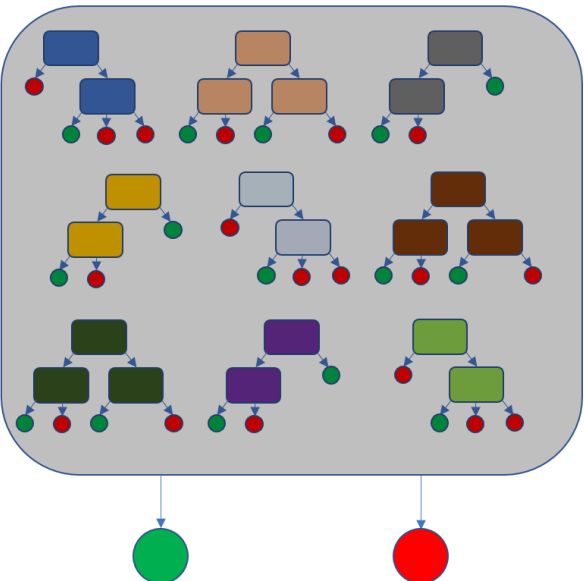
# Train the RandomForestRegressor model
rf_regressor.fit(X_train, y_train)
```

RandomForestRegressor
RandomForestRegressor(random_state=42)

The number of
decision trees in
the random forest



Decision Tree



Random Forest

```
# Calculate and print the R^2 score for the training set
y_pred_train = rf_regressor.predict((X_train))
print("Train set R^2: ", r2_score(y_train, y_pred_train))

# Calculate and print the R^2 score for the test set
y_pred_test = rf_regressor.predict((X_test))
print("Test set R^2: ", r2_score(y_test, y_pred_test))
```

Train set R^2: 0.982779754003236
Test set R^2: 0.8695474438090884

Output training & testing R^2

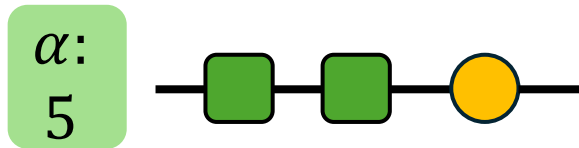
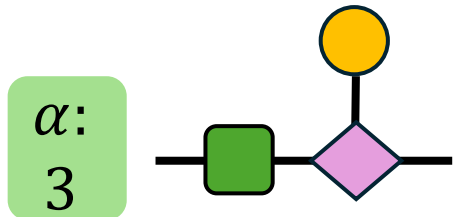
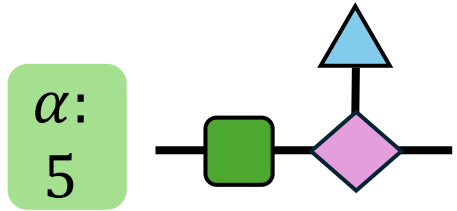
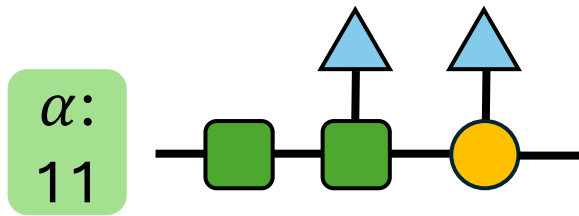
Predict (on training set)

Calculate R^2

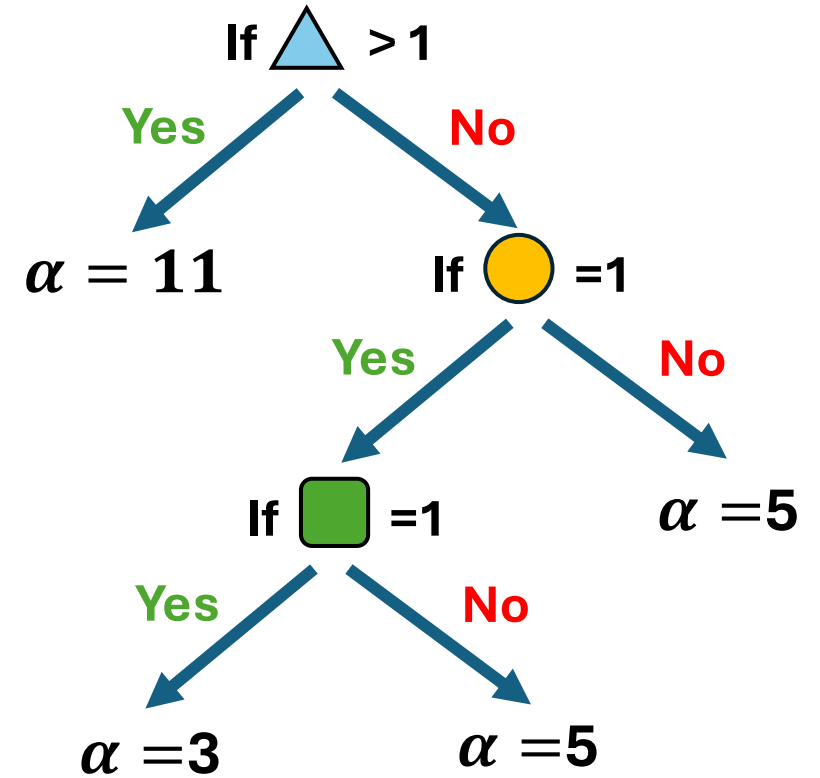
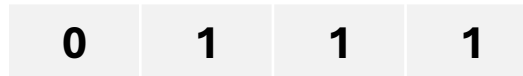
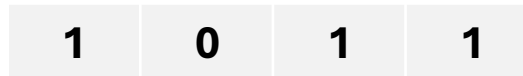
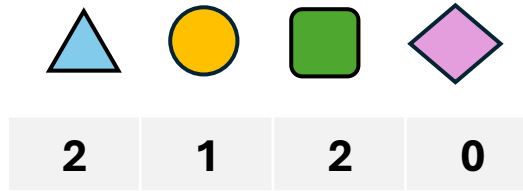
Predict (on testing set)

Calculate R^2

Supervised Learning and Machine Learning Models



Feature
engineering



Random forests

Random seed for initialization of parameters

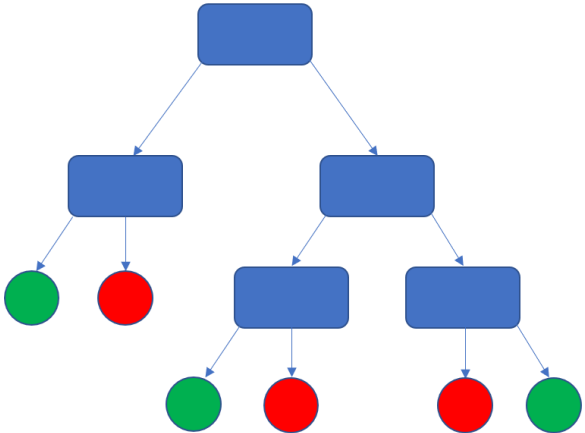
```
from sklearn.ensemble import RandomForestRegressor

# Create a RandomForestRegressor model instance
rf_regressor = RandomForestRegressor(n_estimators=100, random_state=42)

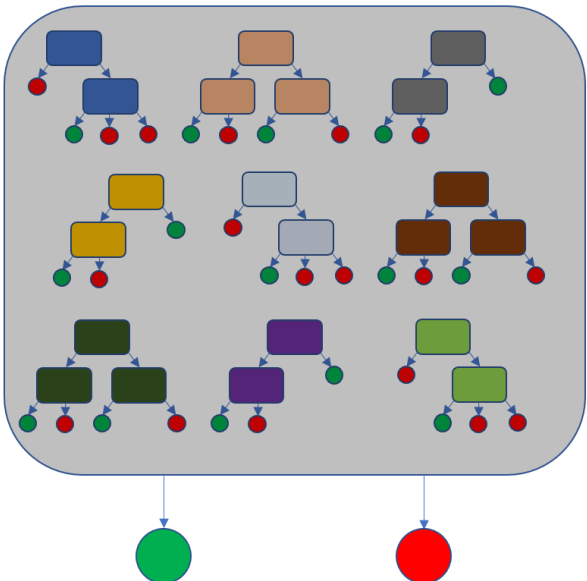
# Train the RandomForestRegressor model
rf_regressor.fit(X_train, y_train)
```

RandomForestRegressor
RandomForestRegressor(random_state=42)

The number of
decision trees in
the random forest



Decision Tree



Random Forest

```
# Calculate and print the R^2 score for the training set
y_pred_train = rf_regressor.predict((X_train))
print("Train set R^2: ", r2_score(y_train, y_pred_train))

# Calculate and print the R^2 score for the test set
y_pred_test = rf_regressor.predict((X_test))
print("Test set R^2: ", r2_score(y_test, y_pred_test))
```

Train set R^2: 0.982779754003236
Test set R^2: 0.8695474438090884

Output training & testing R^2

Random forests

```
plt.figure(figsize=(5,5))

ax=plt.subplot(1, 1, 1)
# Plot the predicted vs. actual values for the training set in blue
plt.plot(y_train, y_pred_train, 'b.')
# Plot the predicted vs. actual values for the test set in green
plt.plot(y_test, y_pred_test, 'g.')
# Set the labels for the x-axis and y-axis
plt.xlabel("Experiment (°C)", fontsize=12)
plt.ylabel("Prediction (°C)", fontsize=12)
# Determine the range for the x-axis and y-axis based on the training data
x0, x1 = min(y_train), max(y_train)
length = 750
x_start, x_end = x0*0.9, x1*1.1
# Set the font size and weight for the x and y ticks
plt.xticks(fontsize=12, fontweight='normal')
plt.yticks(fontsize=12, fontweight='normal')
plt.title("Tg", fontsize=12, fontweight='normal')
plt.gca().set_aspect("equal", adjustable="box")
# Plot the unit line (a line with a slope of 1) in red
plt.plot(np.arange(x_start, x_end, (x_end - x_start) / 10),
np.arange(x_start, x_end, (x_end - x_start) / 10), 'r-')
# Add text annotations to display the R^2 scores for the training and test sets
plt.text(x_start + (x_end - x_start)*0, x_end - (x_end - x_start)*0.12, "Train $R^2={:.2f}$".format(r2_score(y_train, y_pred_train)),{'color':"b"}, fontsize=14)
plt.text(x_start + (x_end - x_start)*0.42, x_end - (x_end - x_start)*0.12, "Test $R^2 = {:.2f}$".format(r2_score(y_test, y_pred_test)),{'color':"g"}, fontsize=14)
```

Plot train samples

Plot test samples

Set the labels for axes

Plot the unit line

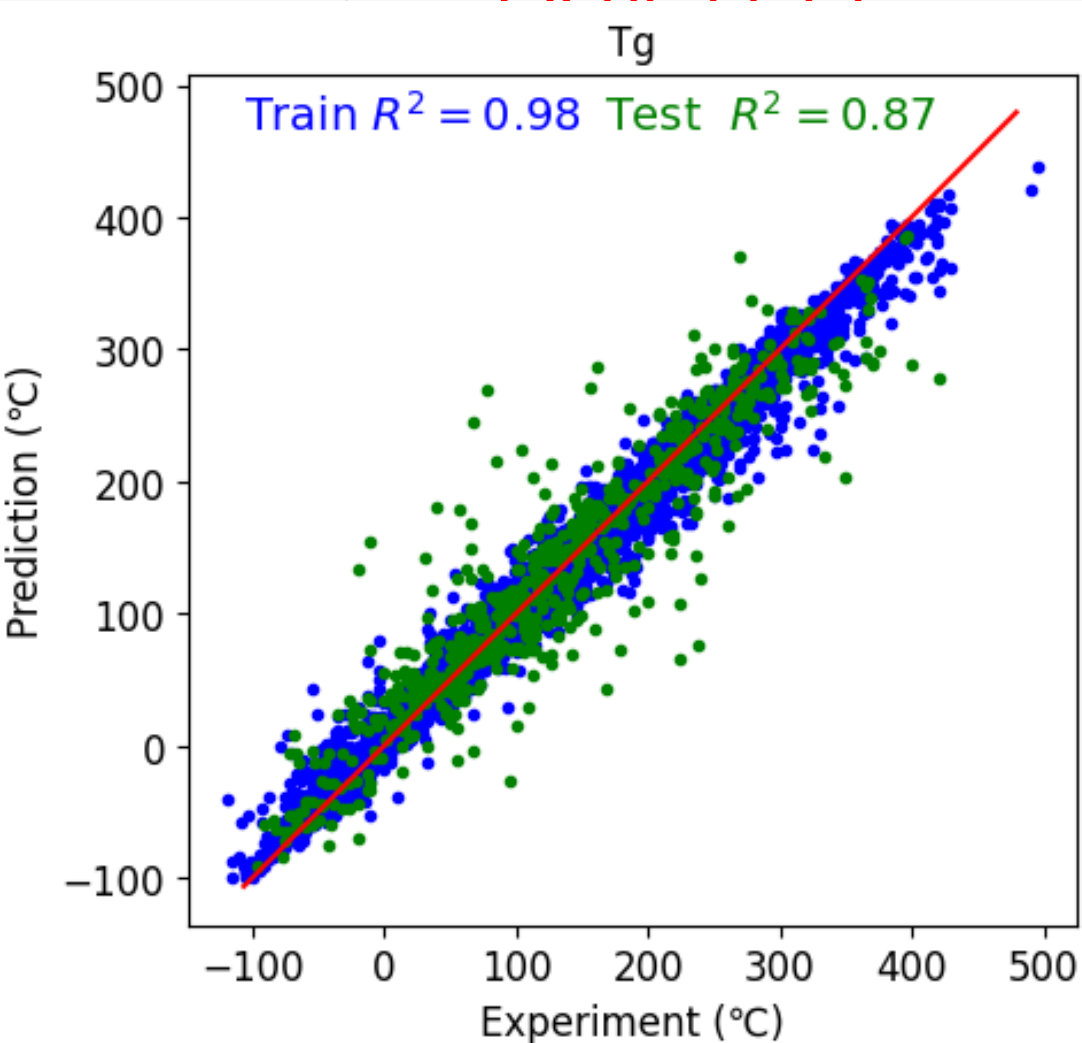
Random forests

```
plt.figure(figsize=(5,5))

ax=plt.subplot(1, 1, 1)
# Plot the predicted vs. actual values for the training set in blue
plt.plot(y_train, y_pred_train, 'b.')
# Plot the predicted vs. actual values for the test set in green
plt.plot(y_test, y_pred_test, 'g.')
# Set the labels for the x-axis and y-axis
plt.xlabel("Experiment (°C)", fontsize=12)
plt.ylabel("Prediction (°C)", fontsize=12)
# Determine the range for the x-axis and y-axis based
x0, x1 = min(y_train), max(y_train)
length = 750
x_start, x_end = x0*0.9, x1*1.1
# Set the font size and weight for the x and y ticks
plt.xticks(fontsize=12, fontweight='normal')
plt.yticks(fontsize=12, fontweight='normal')
plt.title("Tg", fontsize=12, fontweight='normal')
plt.gca().set_aspect("equal", adjustable="box")
# Plot the unit line (a line with a slope of 1) in red
plt.plot(np.arange(x_start, x_end, (x_end - x_start)
np.arange(x_start, x_end, (x_end - x_start) / 10), 'r')
# Add text annotations to display the R^2 scores for
plt.text(x_start + (x_end - x_start)*0, x_end - (x_end - x_start),
plt.text(x_start + (x_end - x_start)*0.42, x_end - (x_end - x_start),
```

Plot train samples

Plot test samples



```
color':"b"}, fontsize=14)
color':"g"}, fontsize=14)
```

Feedforward Neural Network

```
# Set the number of neurons for the four hidden layers.  
n_1=int(256)  
n_2=int(64)  
n_3=int(2048)  
n_4=int(512)
```

Set the number of neurons for hidden layers

```
from keras.layers import Input, Dense  
from keras.models import Model  
from keras.utils import plot_model  
from keras.layers import Concatenate  
from numpy.random import seed  
import tensorflow
```

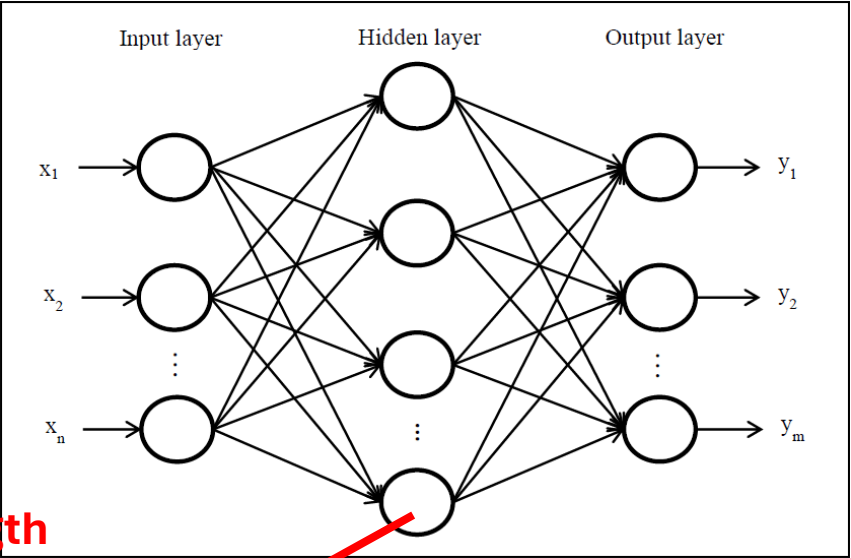
Import deep learning framework from TensorFlow package

```
# Define the input layer with a shape of 1024 features  
A1 = Input(shape=[1024],name='A1')  
# Define the hidden layers  
A2 = Dense(n_1, activation='relu',name='A2')(A1)  
A3 = Dense(n_2, activation='relu',name='A3')(A2)  
A4 = Dense(n_3, activation='relu',name='A4')(A3)  
A5 = Dense(n_4, activation='relu',name='A5')(A4)  
A6 = Dense(1, name='A6')(A5)
```

Input layer, equals the length of the input features

Add hidden layers

Output layer, equals the number of predicting properties



$$a_m^{(l)} = f\left(\sum_{i=1}^n w_i^{(l)} a_i^{(l-1)} + b_i^{(l)}\right)$$

Input

Layer name

```
A2 = Dense(n_1, activation='relu',name='A2')(A1)
```

Number of neurons

Activation function (Sigmoid, Tanh, ReLU, Softmax)

Feedforward Neural Network

Set the number of neurons for the four hidden layers.

```
n_1=int(256)
n_2=int(64)
n_3=int(2048)
n_4=int(512)
```

Set the number of neurons for hidden layers

```
from keras.layers import Input, Dense
from keras.models import Model
from keras.utils import plot_model
from keras.layers import Concatenate
from numpy.random import seed
import tensorflow
```

Import deep learning framework from TensorFlow package

Define the input layer with a shape of 1024 features

```
A1 = Input(shape=[1024], name='A1')
```

Define the hidden layers

```
A2 = Dense(n_1, activation='relu', name='A2')(A1)
```

```
A3 = Dense(n_2, activation='relu', name='A3')(A2)
```

```
A4 = Dense(n_3, activation='relu', name='A4')(A3)
```

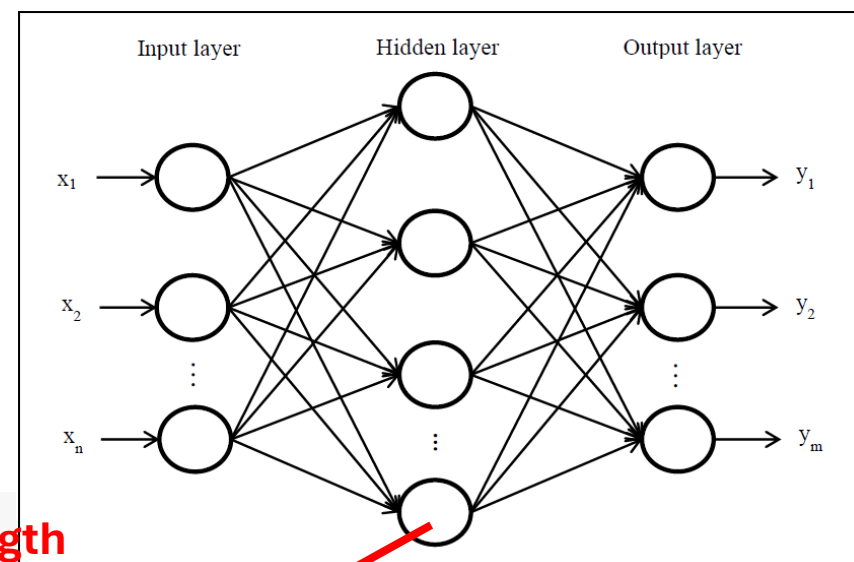
```
A5 = Dense(n_4, activation='relu', name='A5')(A4)
```

```
A6 = Dense(1, name='A6')(A5)
```

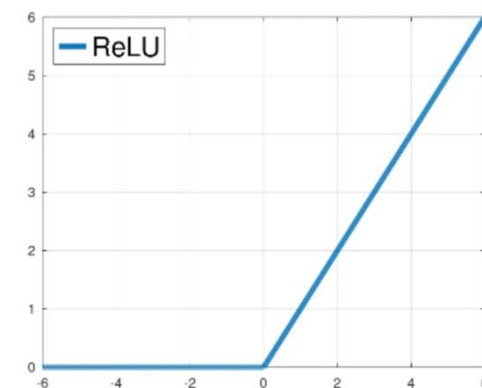
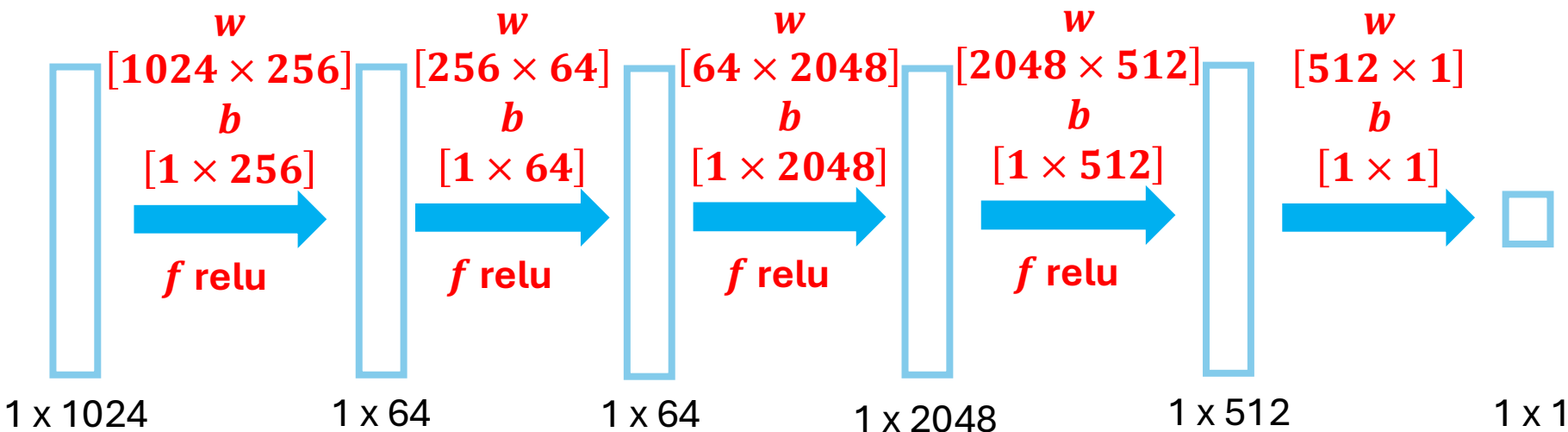
Input layer, equals the length of the input features

Add hidden layers

Output layer, equals the number of predicting properties



$$a_m^{(l)} = f\left(\sum_{i=1}^n w_i^{(l)} a_i^{(l-1)} + b_i^{(l)}\right)$$

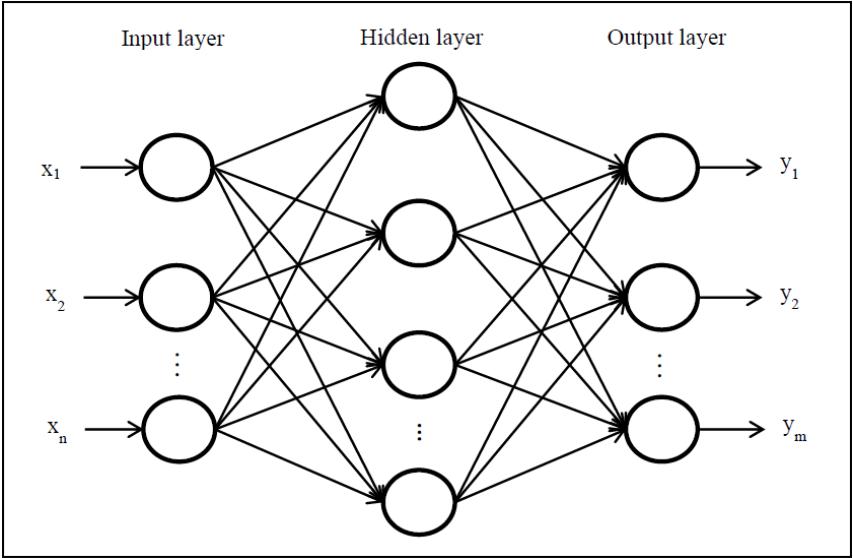


$$f(x) = \max(0, x)$$

Feedforward Neural Network

```
# Set the number of neurons for the four hidden layers.  
n_1=int(256)  
n_2=int(64)  
n_3=int(2048)  
n_4=int(512)
```

```
from keras.layers import Input, Dense  
from keras.models import Model  
from keras.utils import plot_model  
from keras.layers import Concatenate  
from numpy.random import seed  
import tensorflow
```



```
✓ 16m ▶ # Define the input layer with a shape of 1024 features  
A1 = Input(shape=[1024],name='A1')  
# Define the hidden layers  
A2 = Dense(n_1, activation='relu',name='A2')(A1)  
A3 = Dense(n_2, activation='relu',name='A3')(A2)  
A4 = Dense(n_3, activation='relu',name='A4')(A3)  
A5 = Dense(n_4, activation='relu',name='A5')(A4)  
A6 = Dense(1, name='A6')(A5)  
# Compile the model with Mean Squared Error (MSE) loss function and Adam optimizer  
model = Model(inputs=[A1],outputs=[A6])  
model.compile(loss = "mse", optimizer = 'adam')  
# Set the random seed for reproducibility  
seed(1)  
tensorflow.random.set_seed(1*7+333)  
# Train the model on the training data for 200 epochs with a batch size of 32  
history = model.fit(X_train, y_train, epochs=200, batch_size=32, validation_data = ((X_test), y_test), verbose=0)
```

Assemble the model

Set the loss function and training optimizer

Random seed for initialization of parameters

Information displayed during training

- =0 → no output
- =1 → progress bar
- =2 → one line per epoch

Train the model

Train set

Test set

The number of training samples processed before the model updates its parameters once

Training time

The number of times the model trains on the entire training dataset

Feedforward Neural Network

```
# Calculate and print the R^2 score for the training set
y_pred_train = model.predict((X_train))
print("Train set R^2: ", r2_score(y_train, y_pred_train))
# Calculate and print the R^2 score for the test set
y_pred_test = model.predict((X_test))
print("Test set R^2: ", r2_score(y_test, y_pred_test))
```

Predict (on training set)

Calculate R^2

Predict (on testing set)

Calculate R^2

```
195/195 ————— 2s 8ms/step
Train set R^2:  0.9959673342464838
22/22 ————— 0s 8ms/step
Test set R^2:  0.9023972176641257
```

Output training & testing R^2

```
import matplotlib.pyplot as plt
```

```
# Extract loss values for training and validation sets
train_loss = history.history['loss']
val_loss = history.history['val_loss']
```

Obtain the loss values during training

```
# Plotting the loss curves
plt.figure(figsize=(12, 6))
plt.plot(train_loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training and Validation Loss over Epochs for the $T_g$ model')
plt.legend()
plt.grid(True)
plt.show()
```

Plot the loss values during training

Feedforward Neural Network

```
# Calculate and print the R^2 score for the training set
```

```
y_pred_train = model.predict(X_train)
```

```
print("Train set R^2: %f" % r2_train)
```

```
# Calculate and print the R^2 score for the test set
```

```
y_pred_test = model.predict(X_test)
```

```
print("Test set R^2: %f" % r2_test)
```

```
195/195 —————
```

```
Train set R^2: 0.9999
```

```
22/22 —————
```

```
Test set R^2: 0.9999
```

```
import matplotlib.pyplot as plt
```

```
# Extract loss values from the history object
```

```
train_loss = history.history['loss']
```

```
val_loss = history.history['val_loss']
```

```
# Plotting the loss over epochs
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(train_loss, label='Training Loss')
```

```
plt.plot(val_loss, label='Validation Loss')
```

```
plt.xlabel('Epochs')
```

```
plt.ylabel('Loss')
```

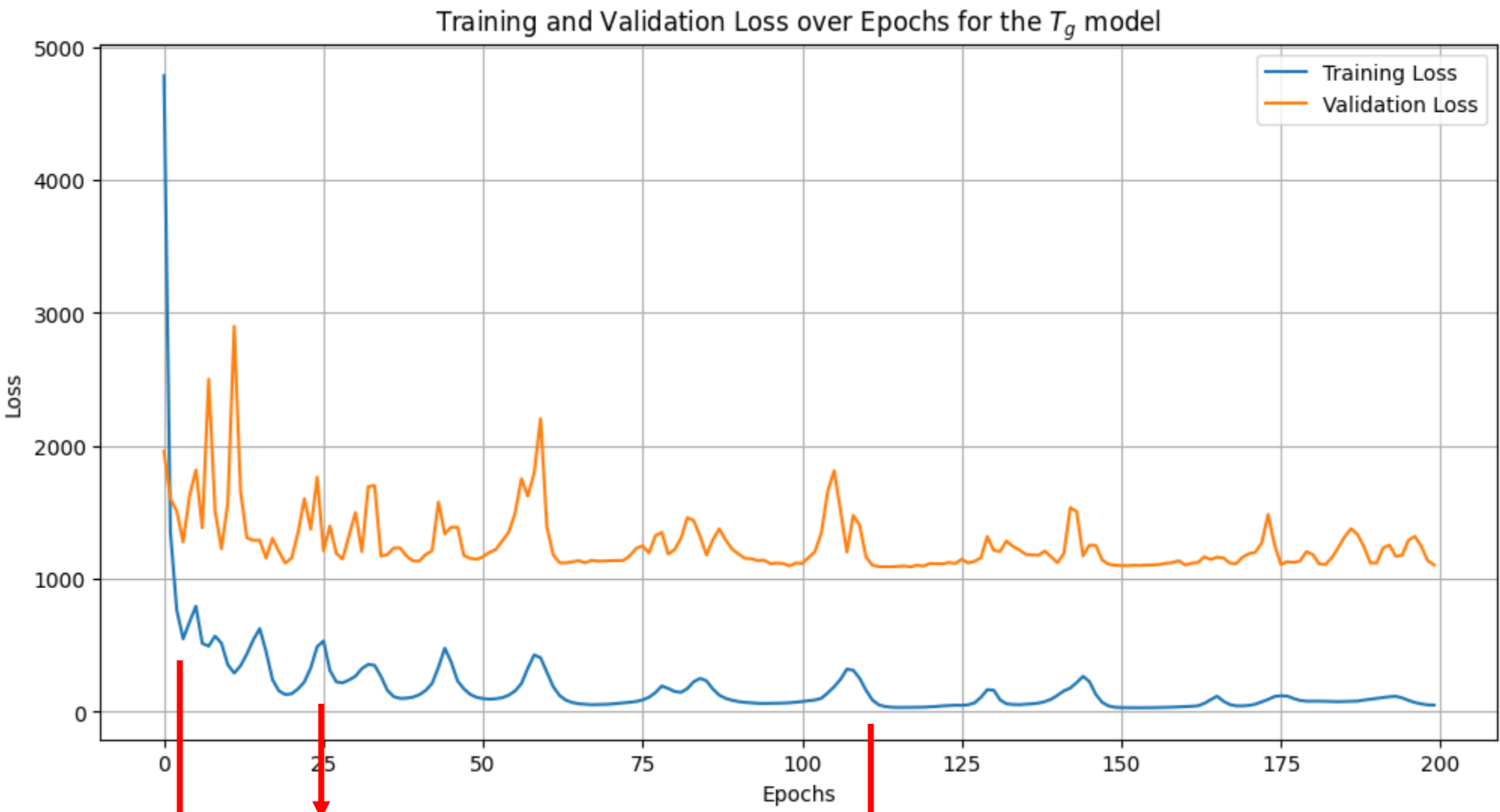
```
plt.title('Training and Validation Loss over Epochs for the  $T_g$  model')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

~~Predict (on training set)~~



Fast decrease

Slow decrease

Fluctuation

Feedforward Neural Network

```
plt.figure(figsize=(5,5))

ax=plt.subplot(1, 1, 1)
# Plot the predicted vs. actual values for the training set in blue
plt.plot(y_train, y_pred_train, 'b.')
# Plot the predicted vs. actual values for the test set in green
plt.plot(y_test, y_pred_test, 'g.')
# Set the labels for the x-axis and y-axis
plt.xlabel("Experiment (°C)", fontsize=12)
plt.ylabel("Prediction (°C)", fontsize=12)
# Determine the range for the x-axis and y-axis based on the training data
x0, x1 = min(y_train), max(y_train)
length = 750
x_start, x_end = x0*0.9, x1*1.1
# Set the font size and weight for the x and y ticks
plt.xticks(fontsize=12, fontweight='normal')
plt.yticks(fontsize=12, fontweight='normal')
plt.title("Tg", fontsize=12, fontweight='normal')
plt.gca().set_aspect("equal", adjustable="box")
# Plot the unit line (a line with a slope of 1) in red
plt.plot(np.arange(x_start, x_end, (x_end - x_start) / 10),
np.arange(x_start, x_end, (x_end - x_start) / 10), 'r-')
# Add text annotations to display the R^2 scores for the training and test sets
plt.text(x_start + (x_end - x_start)*0, x_end - (x_end - x_start)*0.12, "Train $R^2={:.2f}$".format(r2_score(y_train, y_pred_train)),{'color':"b"}, fontsize=14)
plt.text(x_start + (x_end - x_start)*0.42, x_end - (x_end - x_start)*0.12, "Test $R^2 = {:.2f}$".format(r2_score(y_test, y_pred_test)),{'color':"g"}, fontsize=14)
```

Plot train samples

Plot test samples

Set the labels for axes

Plot the unit line

Feedforward Neural Network

```
plt.figure(figsize=(5,5))
```

```
ax=plt.subplot(1, 1, 1)
```

```
# Plot the predicted vs. actual values for the training set in blue
```

```
plt.plot(y_train, y_pred_train, 'b.')
```

```
# Plot the predicted vs. actual values for the test set in green
```

```
plt.plot(y_test, y_pred_test, 'g.')
```

```
# Set the labels for the x-axis and y-axis
```

```
plt.xlabel("Experiment (°C)", fontsize=12)
```

```
plt.ylabel("Prediction (°C)", fontsize=12)
```

```
# Determine the range for the x-axis and y-axis based on the training data
```

```
x0, x1 = min(y_train), max(y_train)
```

```
length = 750
```

```
x_start, x_end = x0*0.9, x1*1.1
```

```
# Set the font size and weight for the x and y ticks
```

```
plt.xticks(fontsize=12, fontweight='normal')
```

```
plt.yticks(fontsize=12, fontweight='normal')
```

```
plt.title("Tg", fontsize=12, fontweight='normal')
```

```
plt.gca().set_aspect("equal", adjustable="box")
```

```
# Plot the unit line (a line with a slope of 1) in red
```

```
plt.plot(np.arange(x_start, x_end, (x_end - x_start) / 10),
```

```
np.arange(x_start, x_end, (x_end - x_start) / 10), 'r-')
```

```
# Add text annotations to display the R^2 scores for the training and test sets
```

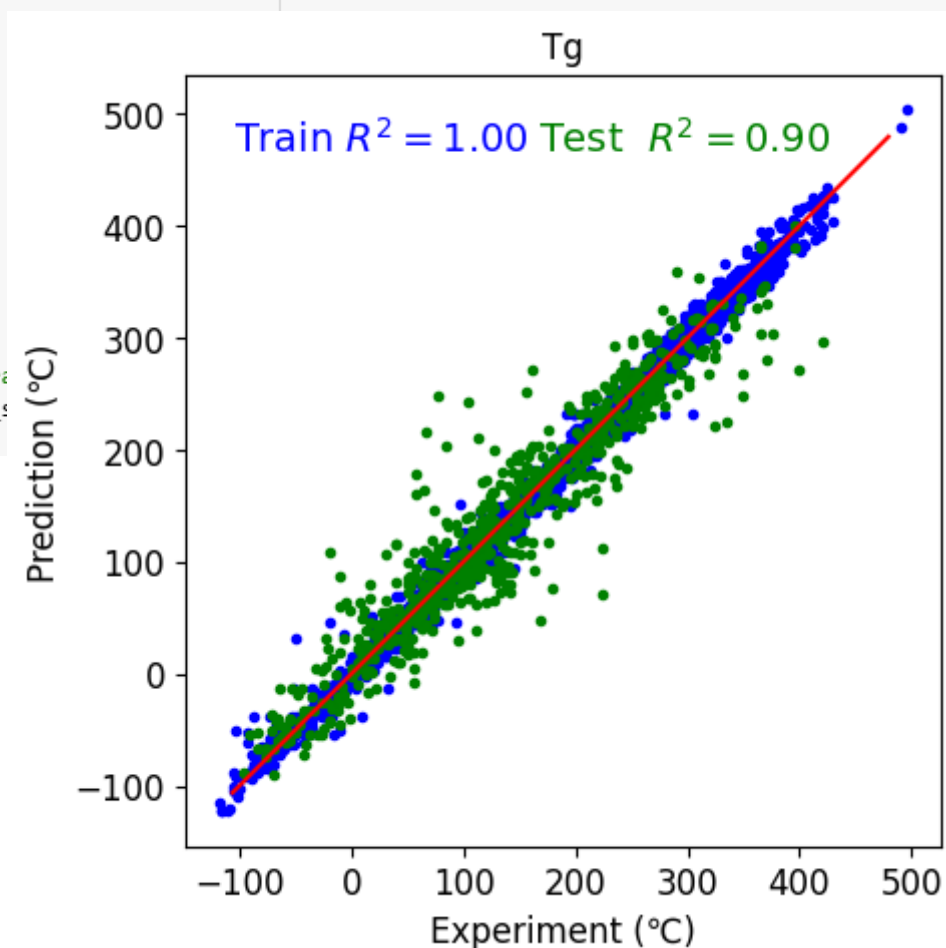
```
plt.text(x_start + (x_end - x_start)*0, x_end - (x_end - x_start)*0,
```

```
plt.text(x_start + (x_end - x_start)*0.42, x_end - (x_end - x_start)*0.42,
```

Plot train samples

Plot test samples

Set the labels for axes



```
,{'color':"b"}, fontsize=14)
```

```
),{'color':"g"}, fontsize=14)
```

Morgan Fingerprint with Frequency as feature engineering method

```
[ ] X = MFF  
    y = Tg_df['Tg(C)']
```

→ The generated Morgan fingerprint with Frequency

→ Column name

```
[ ] # Generate train/test split by reserving 10% of data as test set  
  
test_fraction = 0.1  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_fraction, shuffle=True, random_state = 11)
```

→ Generate train/test split

Fitting and Evaluating an FNN

```
A1 = Input(shape=(1210),name='A1')  
A2 = Dense(n_1, activation='relu',name='A2')(A1)  
A3 = Dense(n_2, activation='relu',name='A3')(A2)  
A4 = Dense(n_3, activation='relu',name='A4')(A3)  
A5 = Dense(n_4, activation='relu',name='A5')(A4)  
A6 = Dense(1, name='A6')(A5)  
  
model = Model(inputs=[A1],outputs=[A6])  
model.compile(loss = "mse", optimizer = 'adam')  
  
seed(1)  
tensorflow.random.set_seed(1*7+333)  
  
history = model.fit(X_train, y_train, epochs=200, batch_size=32, validation_data = ((X_test), y_test), verbose=0)
```

→ Input layer, equals the length of the input features

Descriptors as feature engineering method

```
[ ] X = minmax_descriptors_df
    y = Tg_df['Tg(C)']
```

→ The generated descriptors

→ Column name

```
[ ] # Generate train/test split by reserving 10% of data as test set

test_fraction = 0.1
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_fraction, shuffle=True, random_state = 11)
```

→ Generate train/test split

Fitting and Evaluating an FNN

```
A1 = Input(shape=([148]),name='A1')
A2 = Dense(n_1, activation='relu',name='A2')(A1)
A3 = Dense(n_2, activation='relu',name='A3')(A2)
A4 = Dense(n_3, activation='relu',name='A4')(A3)
A5 = Dense(n_4, activation='relu',name='A5')(A4)
A6 = Dense(1, name='A6')(A5)

model = Model(inputs=[A1],outputs=[A6])
model.compile(loss = "mse", optimizer = 'adam')

seed(1)
tensorflow.random.set_seed(1*7+333)

history = model.fit(X_train, y_train, epochs=200, batch_size=4, validation_data = ((X_test), y_test), verbose=0)
```

→ Input layer, equals the length of the input features

Benchmark

```
new_data = {
    'Feature Engineering': 'Descriptors',
    'Model': 'FNN',
    'Train R^2': [r2_score(y_train, y_pred_train)],
    'Test R^2': [r2_score(y_test, y_pred_test)]
}
new_row_df = pd.DataFrame(new_data)
r2_df = pd.concat([r2_df, new_row_df], ignore_index=True)
r2_df
```

Record Feature engineering method

Record sML model

Record R^2

	Feature Engineering	Model	Train R^2	Test R^2
0	Morgan Fingerprint	Lasso (alpha=1)	0.734657	0.723019
1	Morgan Fingerprint	Lasso (alpha=0.02)	0.874686	0.830296
2	Morgan Fingerprint	RF	0.982780	0.869547
3	Morgan Fingerprint	FNN	0.989435	0.895693
4	Morgan Fingerprint with Frequency	Lasso	0.743475	0.750664
5	Morgan Fingerprint with Frequency	RF	0.985558	0.894733
6	Morgan Fingerprint with Frequency	FNN	0.992221	0.880303
7	Descriptors	Lasso	0.659180	0.656326
8	Descriptors	RF	0.981981	0.877431
9	Descriptors	FNN	0.960202	0.882060

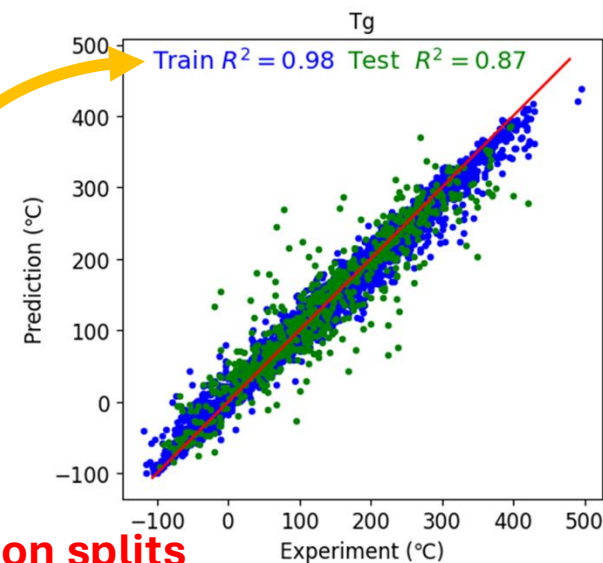
Highest test R^2

Cross-validation strategy

```
[ ] # Generate train/test split by reserving 10% of data as test set
```

```
test_fraction = 0.1
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_fraction, shuffle=True, random_state = 11)
```



```
from sklearn.model_selection import KFold, RepeatedKFold
# Initialize Repeated K-Fold cross-validation with 5 splits and 2 repeats
# Set the random_state to 42 for reproducibility
kfold = RepeatedKFold(n_splits=5, random_state=42, n_repeats=2)
```

Generate cross-validation splits

Random seed

**The number of folds
(parts) the dataset is split**

**How many times the whole
k-fold process is repeated
with different random splits**

Cross-validation strategy

```
import time
# Start the timer to measure the time taken for cross-validation
tic = time.perf_counter()
# Initialize
fold_no = 1
average_r2_train = 0
average_r2_test = 0
# Perform Repeated K-Fold cross-validation
for train_index, test_index in kfold.split(X):
    print(f'Training fold {fold_no}...')

    # Split the data into training and testing sets for the current fold
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]
    # Define the neural network architecture
    A1 = Input(shape=[148], name='A1')
    A2 = Dense(n_1, activation='relu', name='A2')(A1)
    A3 = Dense(n_2, activation='relu', name='A3')(A2)
    A4 = Dense(n_3, activation='relu', name='A4')(A3)
    A5 = Dense(n_4, activation='relu', name='A5')(A4)
    A6 = Dense(1, name='A6')(A5)
    # Create and compile the model
    model = Model(inputs=[A1], outputs=[A6])
    model.compile(loss="mse", optimizer='adam')
    # Set the random seed for reproducibility
    seed(1)
    tensorflow.random.set_seed(1*7+333)
    # Train the model on the training data for the current fold
    history = model.fit(X_train, y_train, epochs=200, batch_size=32, validation_data=((X_test), y_test), verbose=0)
    # Calculate and print the R^2 score for the training and testing set for the current fold
    y_pred_train = model.predict((X_train))
    average_r2_train = average_r2_train + r2_score(y_train, y_pred_train)
    y_pred_test = model.predict((X_test))
    average_r2_test = average_r2_test + r2_score(y_test, y_pred_test)
    fold_no += 1

# Stop the timer after cross-validation is complete
toc = time.perf_counter()
# Print the average R^2 scores for both train and test sets across all folds
print('Average Train set R^2:', average_r2_train/10)
print('Average Test set R^2:', average_r2_test/10)
print(f"Completed in {toc - tic:0.3f} seconds.")
```

Split train/test set in different way

Assemble the model

Train the model

Calculate the average R^2

Descriptors as feature engineering method

```
Training fold 1...
173/173 _____ 1s 4ms/step
44/44 _____ 0s 4ms/step
Training fold 2...
173/173 _____ 1s 5ms/step
44/44 _____ 0s 5ms/step
Training fold 3...
173/173 _____ 1s 4ms/step
44/44 _____ 0s 4ms/step
Training fold 4...
173/173 _____ 1s 5ms/step
44/44 _____ 0s 4ms/step
Training fold 5...
173/173 _____ 1s 6ms/step
44/44 _____ 0s 4ms/step
Training fold 6...
173/173 _____ 1s 5ms/step
44/44 _____ 0s 4ms/step
Training fold 7...
173/173 _____ 1s 5ms/step
44/44 _____ 0s 5ms/step
Training fold 8...
173/173 _____ 1s 4ms/step
44/44 _____ 0s 5ms/step
Training fold 9...
173/173 _____ 1s 4ms/step
44/44 _____ 0s 5ms/step
Training fold 10...
173/173 _____ 1s 4ms/step
44/44 _____ 0s 4ms/step
Average Train set R^2: 0.9627133913875315
Average Test set R^2: 0.8527721377770193
Completed in 8783.954 seconds.
```

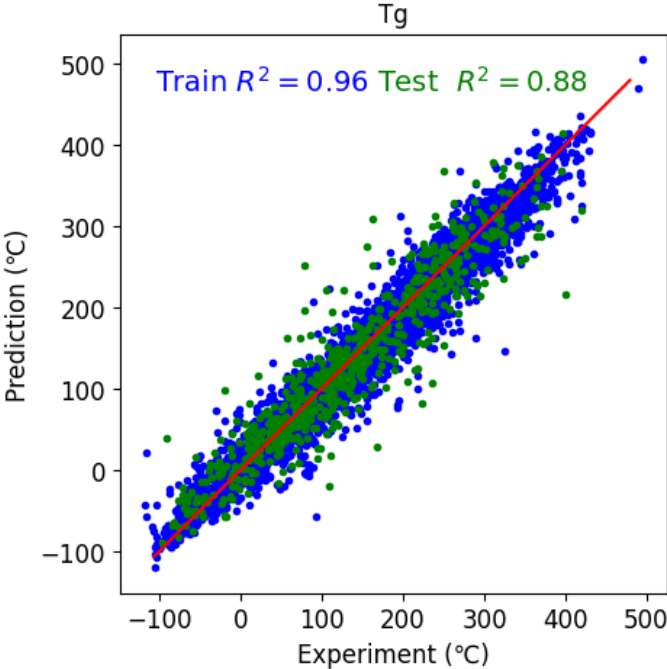
```
from sklearn.model_selection import KFold, RepeatedKFold
# Initialize Repeated K-Fold cross-validation with 5 splits and 2 repeats
# Set the random_state to 42 for reproducibility
kfold = RepeatedKFold(n_splits=5, random_state=42, n_repeats=2)
```

The number of folds
(parts) the dataset is split

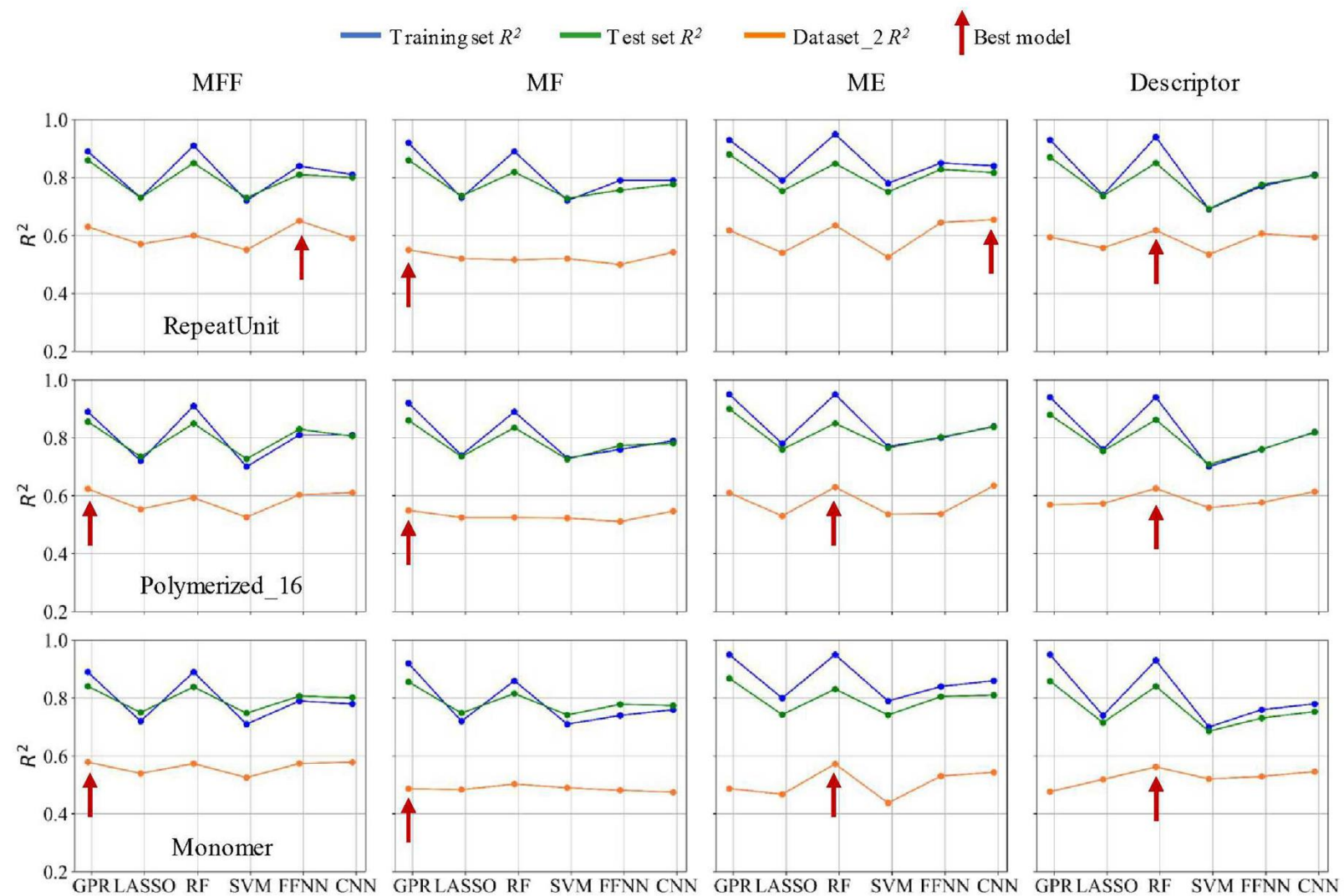
Generate cross-validation splits

The average R^2
Time taken to complete the training
~2.4h

Result with some specific train/test splits



YM_Train R^2	YM_Test R^2	Network	test_size	epochs
0.997523	0.903704	16_128_32_2048	0.1	200
0.997523	0.903704	16_256_128_1024	0.1	100
0.998348	0.903704	128_128_128_32	0.1	200
0.994220	0.896296	16_256_16_256	0.1	100
0.997523	0.896296	16_1024_256_512	0.1	100
0.996697	0.896296	32_32_128_16	0.1	100
0.996697	0.896296	32_2048_16_1024	0.1	200
0.999174	0.896296	64_256_64_512	0.1	200
0.987614	0.896296	64_2048_256_32	0.1	100
0.997523	0.896296	128_512_64_128	0.1	100



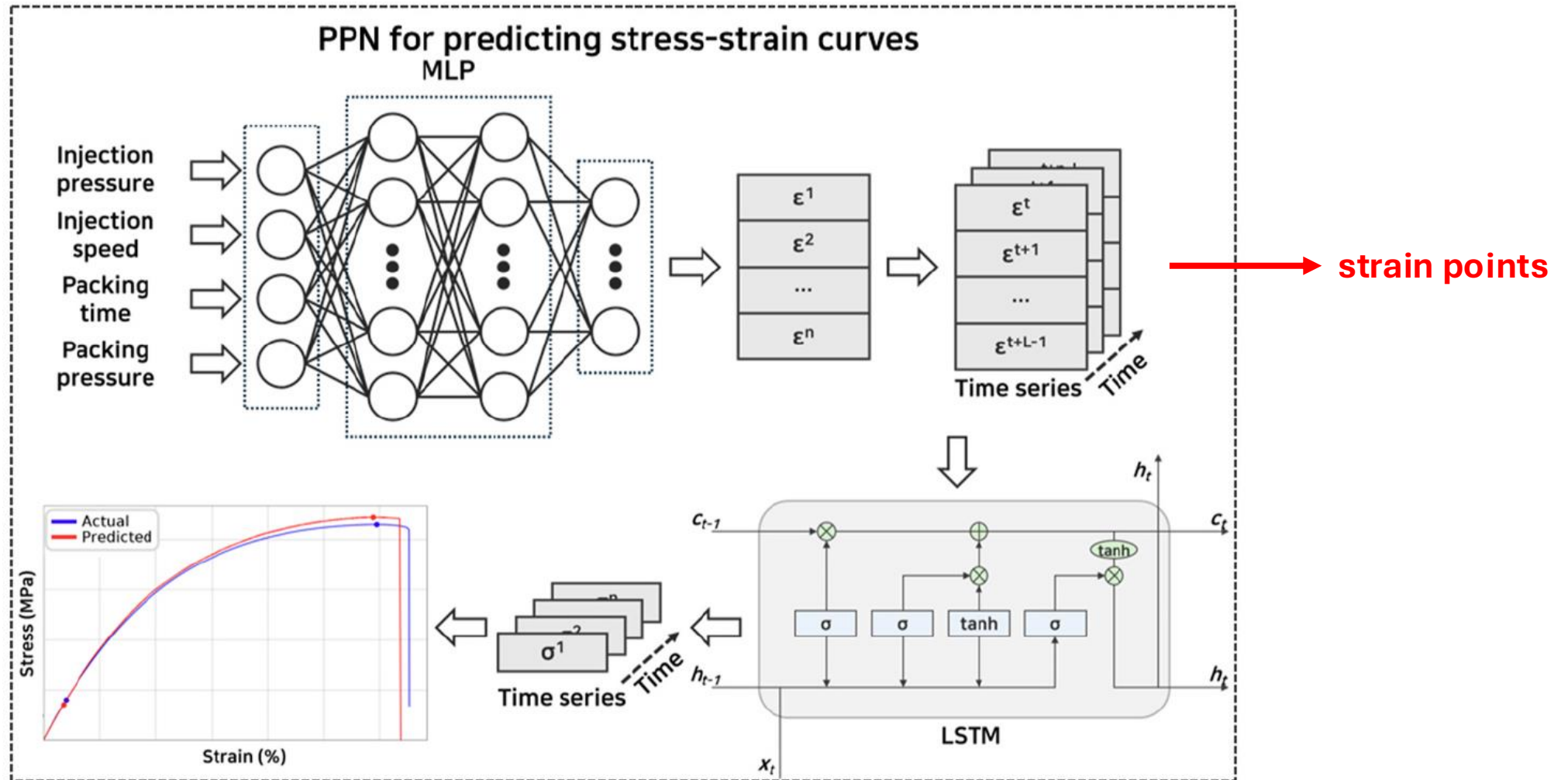
Machine learning-aided network for process-property prediction of injection-molded polyamide-6 composite parts

Taehwan Kim^{a, }, Unghyeon Cho^{a, }, Jinwoo Jang^b, Min-Kyeom Kim^a, Yongjian Fang^a, Jin Young Jung^c, Jun Yeon Hwang^c, Sang Won Lee^{a, *}, Jonghwan Suhr^{a, **, }

^a Department of Mechanical Engineering, Sungkyunkwan University, Seobu-ro 2066, Jangan-gu, Suwon, 16419, Gyeonggi-do, Republic of Korea

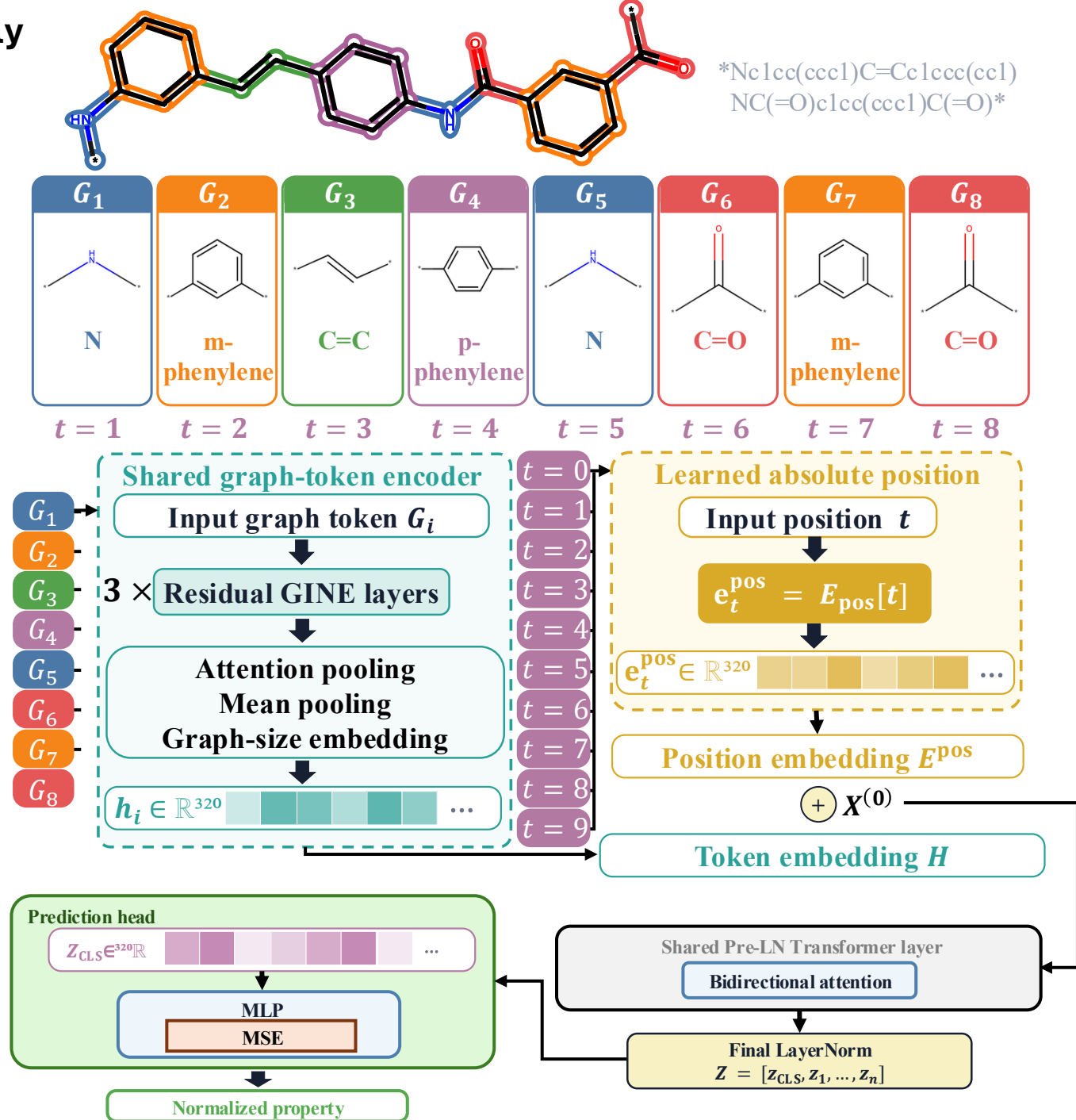
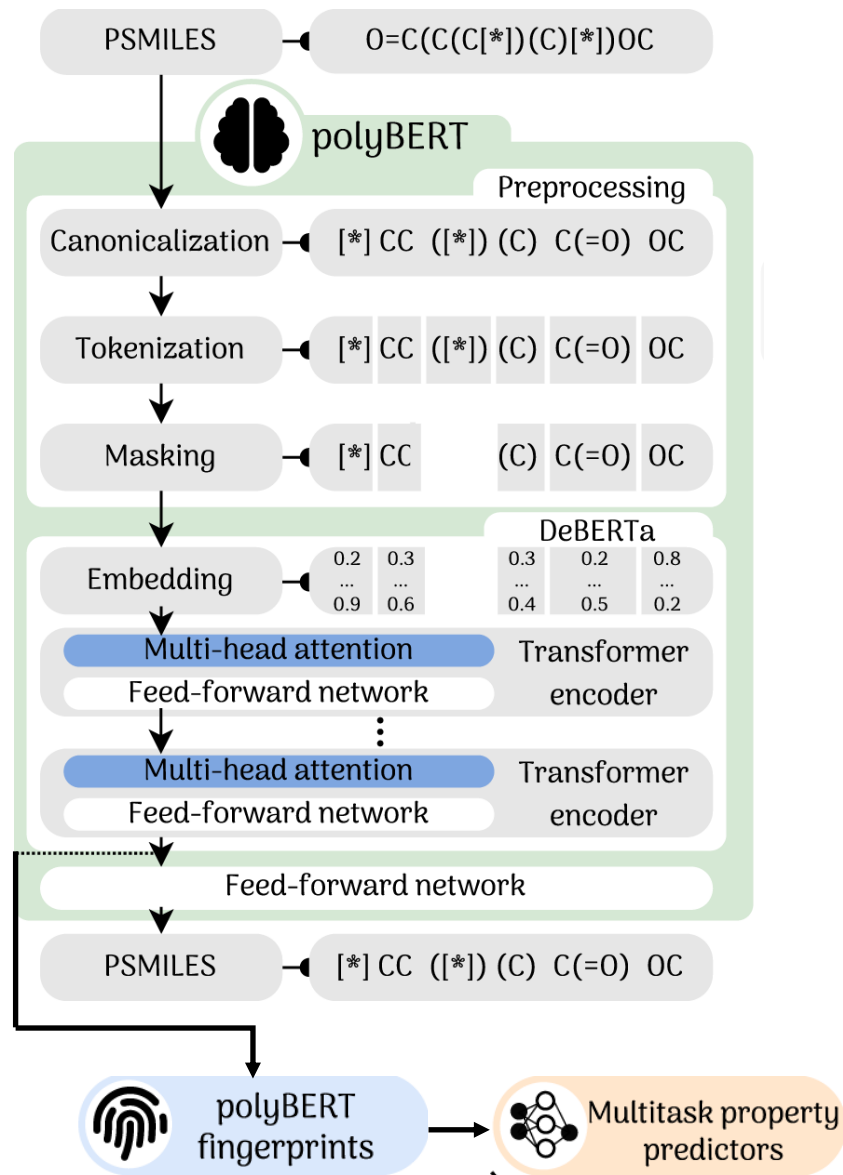
^b Department of Polymer Science, Sungkyunkwan University, Seobu-ro 2066, Jangan-gu, Suwon, 16419, Gyeonggi-do, Republic of Korea

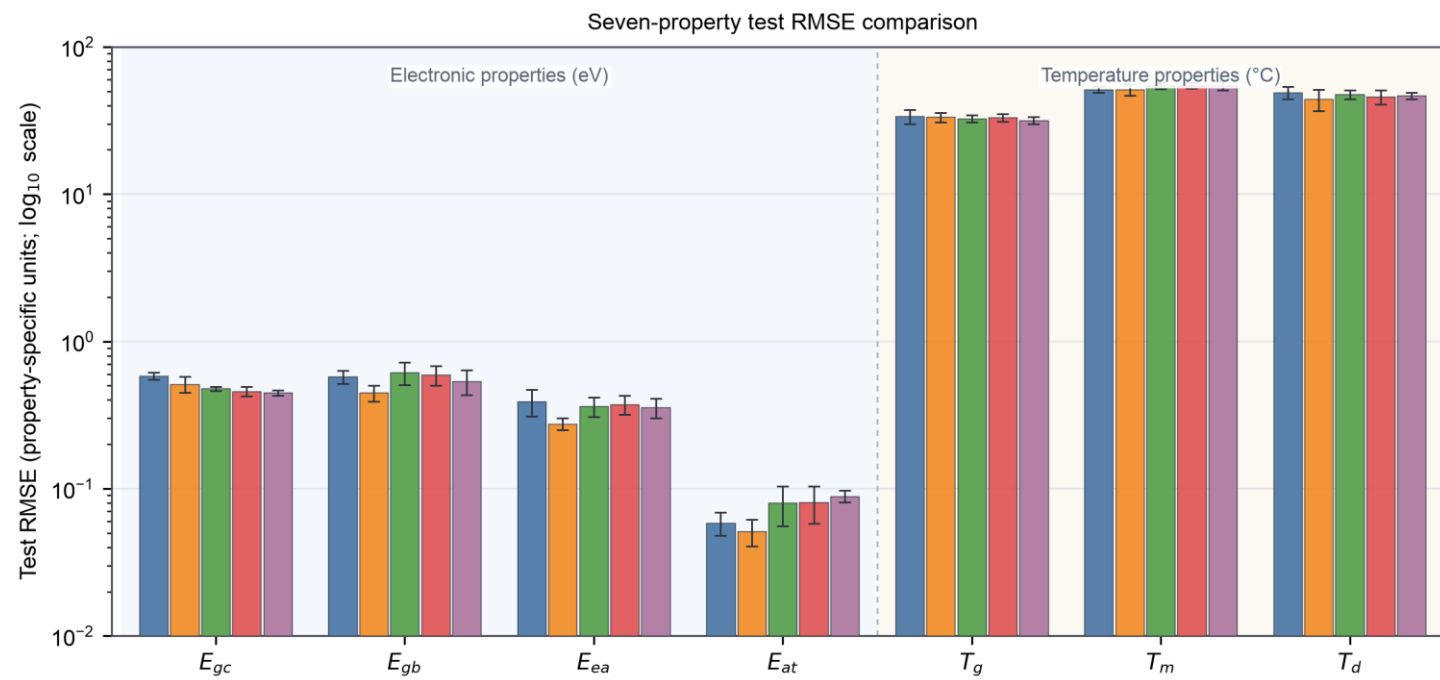
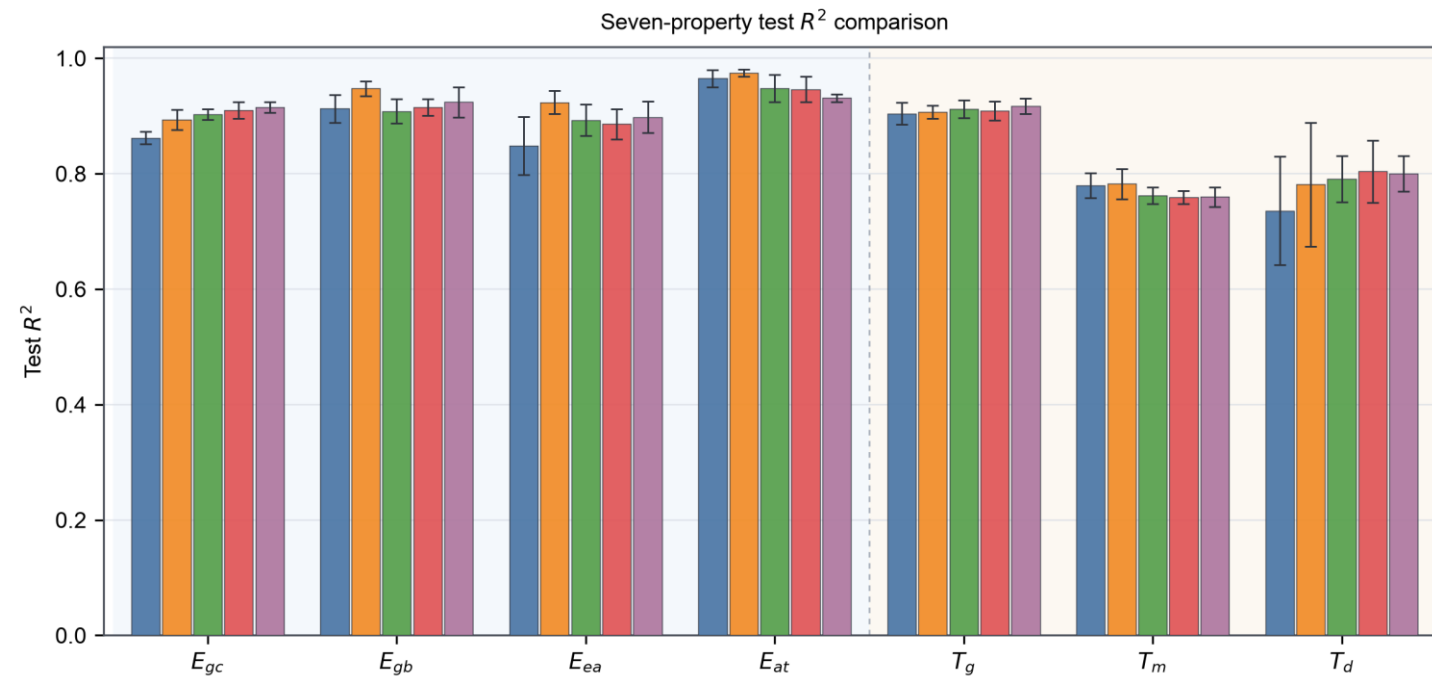
^c Institute of Advanced Composite Materials, Korea Institute of Science and Technology (KIST), Jeonbuk, 55324, Republic of Korea



polyBERT: a chemical language model to enable fully machine-driven ultrafast polymer informatics

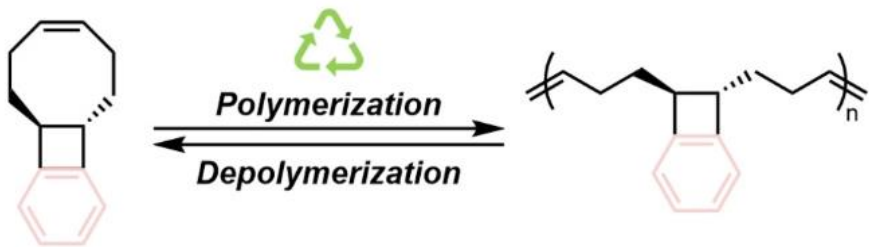
Christopher Kuenneth & Rampi Ramprasad





Small-Scale Testing for Candidate Library Reliability

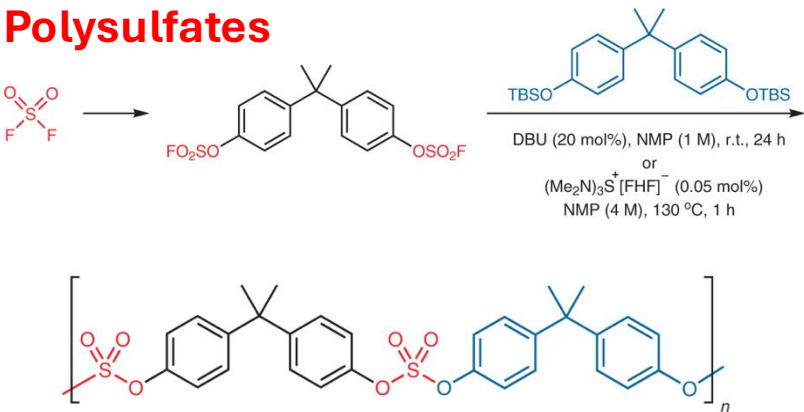
Polycyclooctenes



Junpeng Wang et al.

Model	Train R^2	Test R^2	Candidates R^2
MFF & FNN	0.99	0.82	0.84
MG & GCN	0.97	0.88	0.61

Polysulfates



Polysulfate

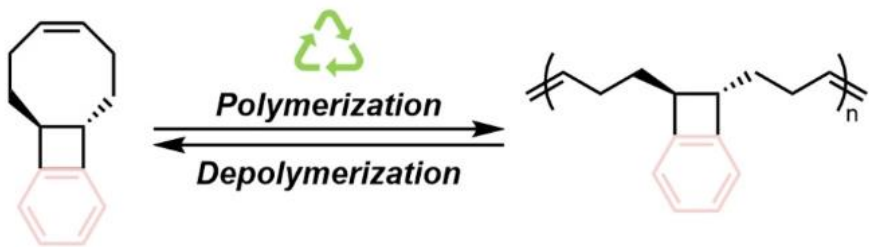
Yi Liu et al.

Model	Train R^2	Test R^2	Candidates R^2
MF & FNN	0.99	0.89	-2.40

- R^2 on the test set reflects model fit but does not fully capture its generalization to the target candidate library.
- The best case is to have a labeled test set within the candidate library, which allows direct evaluation of model suitability.

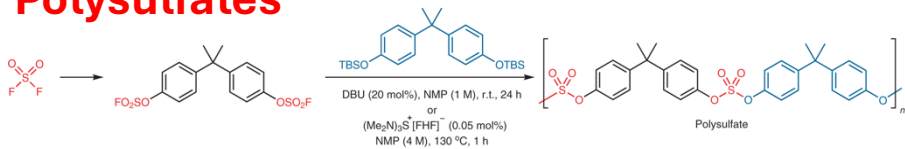
Structure similarity/ Data coverage

Polycyclooctenes

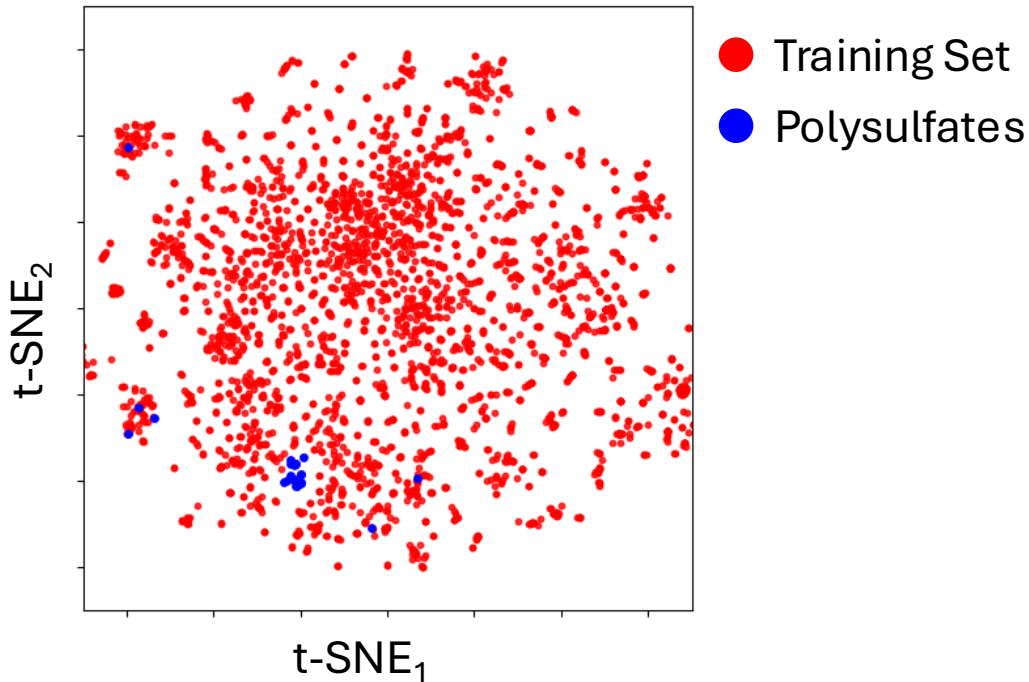
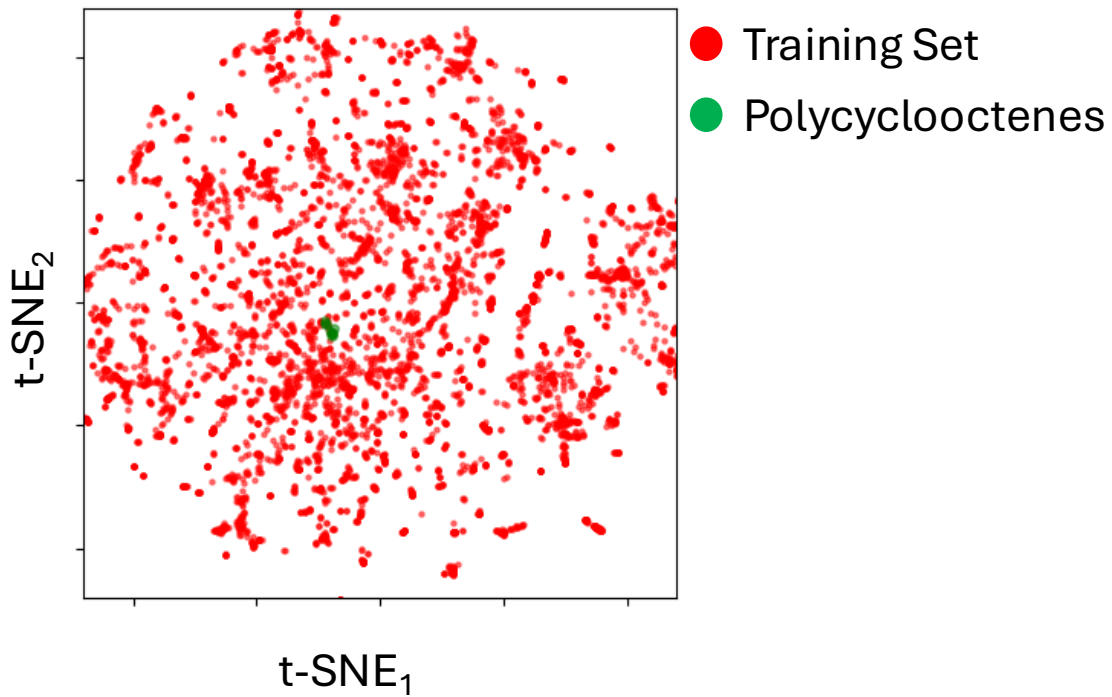


Model	Train R^2	Test R^2	Candidates R^2
MFF & FNN	0.99	0.82	0.84
MG & GCN	0.97	0.88	0.61

Polysulfates



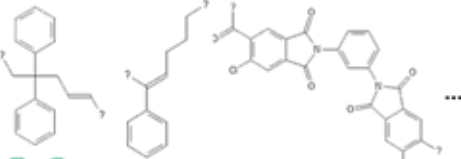
Model	Train R^2	Test R^2	Candidates R^2
MF & FNN	0.99	0.89	-2.40



Data shortage

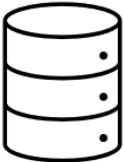


Real Polymer Structures

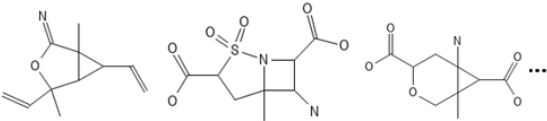


 ~18,000 Real Polymers

V.S.



Small Molecule Compounds



 ~116M Compounds



Property

Count

Log-transformed octanol/water partition coefficient ($\log P$)

14,207

Blood-brain barrier (BBB) permeability

7,807



Property

Count

Glass transition temperature

6,906

Melting temperature

3,633

Decomposition temperature

5,237

Thermal conductivity

68

Heat capacity

79

Property

Count

Young's modulus

923

Yield strength

230

Break strength

982

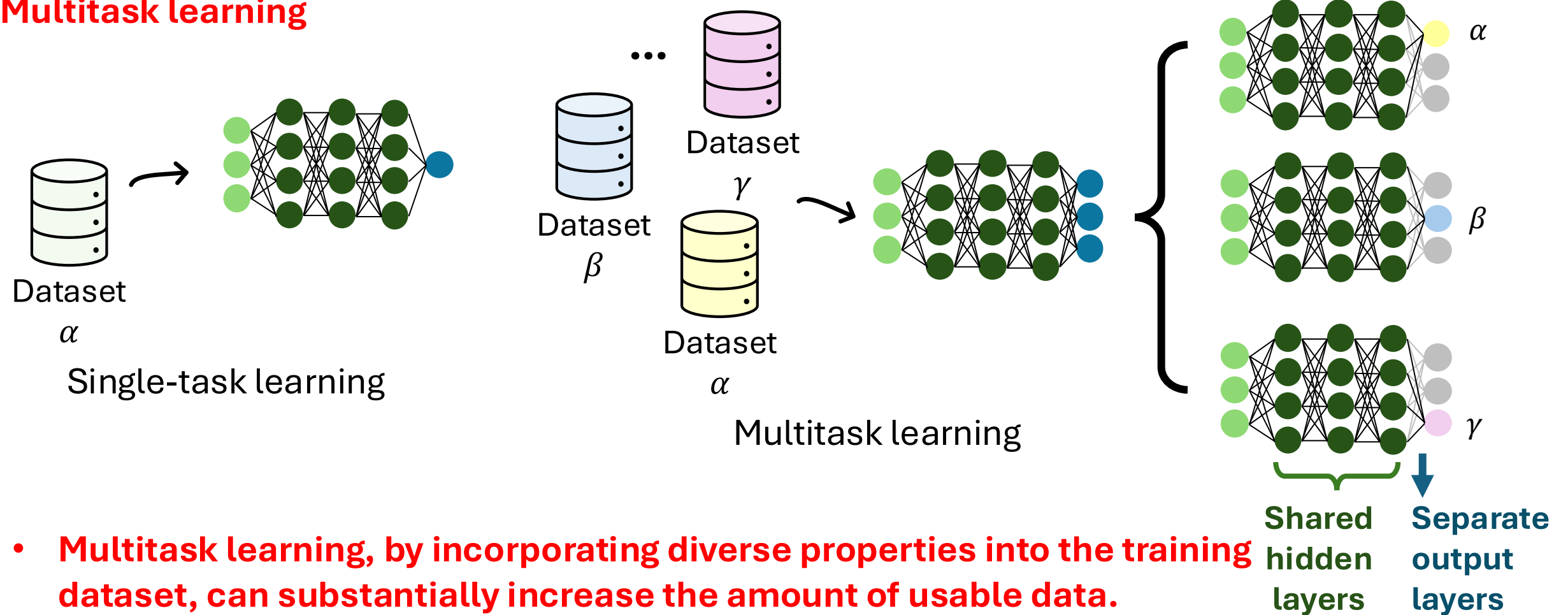
Dielectric constant

627

Bandgap

3,379

Multitask learning



Training Prediction Models with Limited Data

Multitask learning

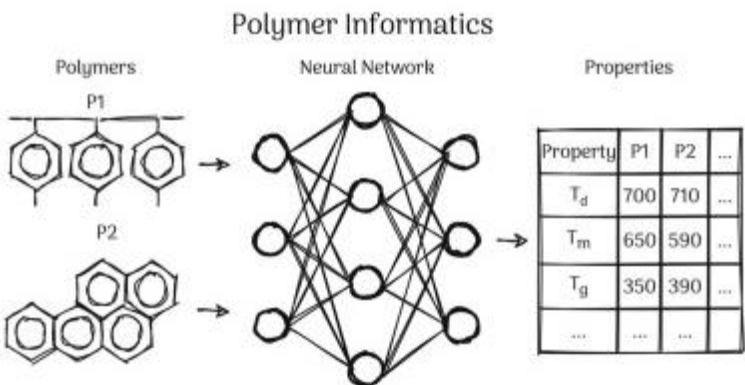
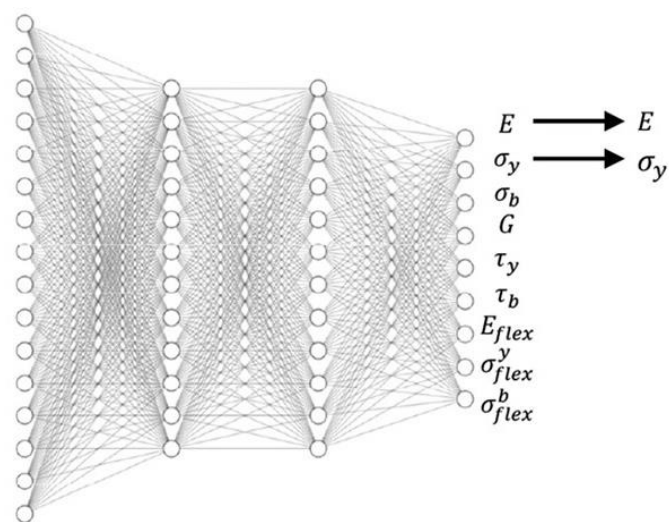


Table 2. Averages of the normalized root mean squared errors values

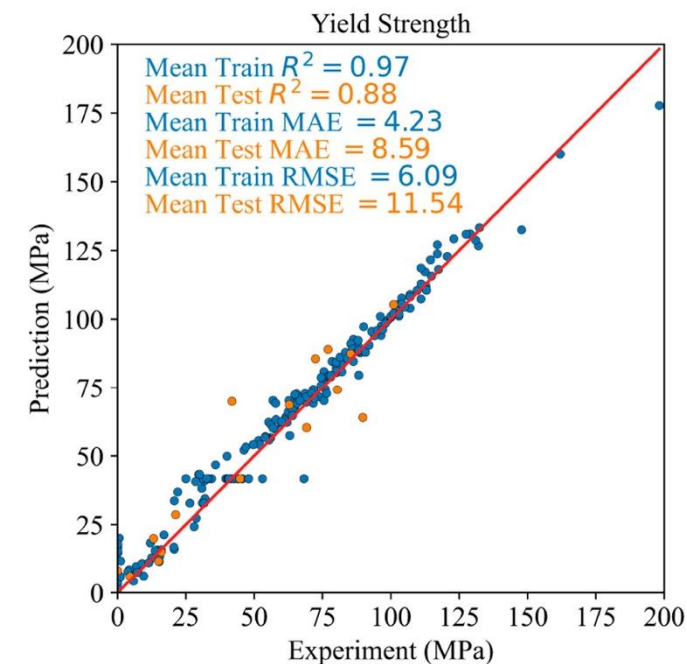
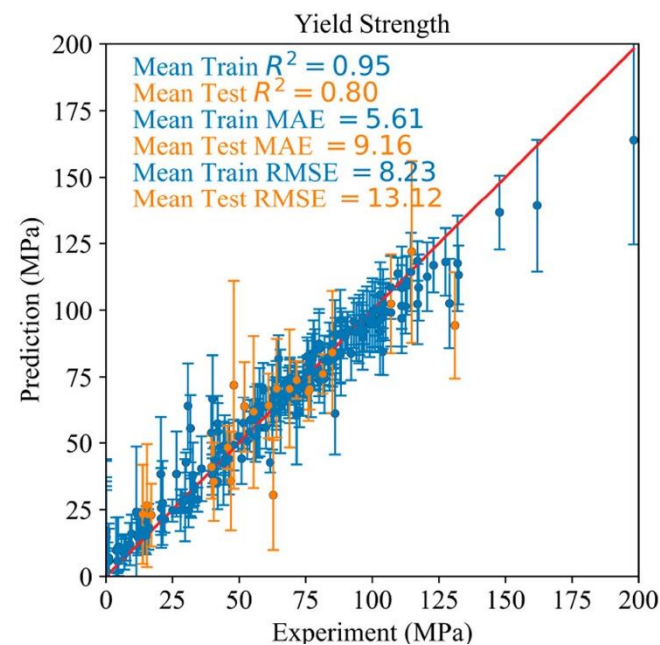
Model	All	Categories					
		Thermal	Thermod. & physical	Electronic	Optical & dielectric	Mech.	Solubility & permeability
GP-ST	0.93	0.92	0.86	0.88	0.98	1.00	0.93
NN-ST	0.89	0.95	0.76	0.91	0.91	0.95	0.94
NN-MT1	0.89	0.98	0.83	0.97	0.83	0.94	0.92
NN-MT2-all	0.79	0.97	0.82	0.96	0.69	0.89	0.70
NN-MT2-sub	0.65	0.94	0.87	0.79	0.47	0.82	0.46

The root mean squared error averages were normalized for each property so that the maximal value is 1.

Kuenneth, C., Rajan, A.C., Tran, H., Chen, L., Kim, C. and Ramprasad, R., 2021. Polymer informatics with multi-task learning. *Patterns*, 2(4).

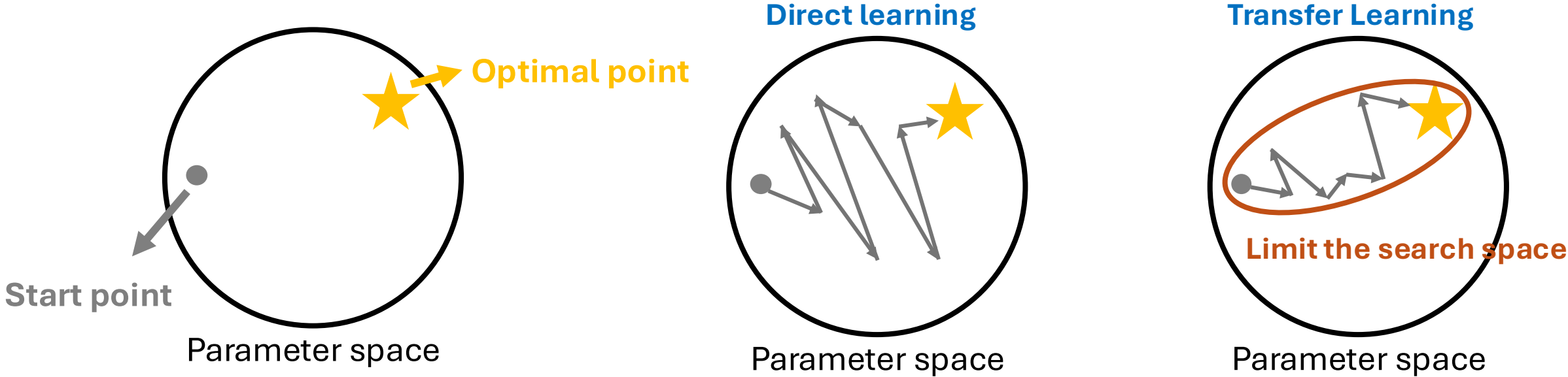
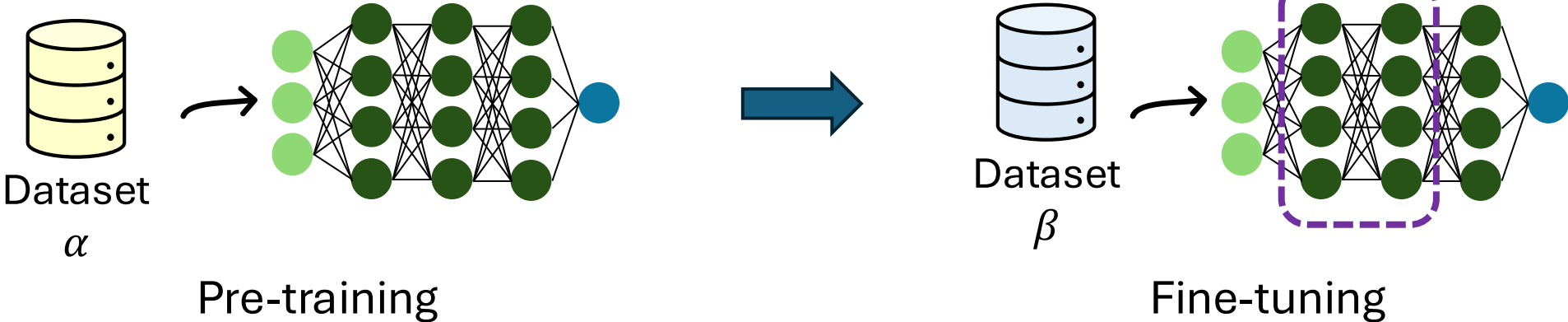


Multi-Task Learning



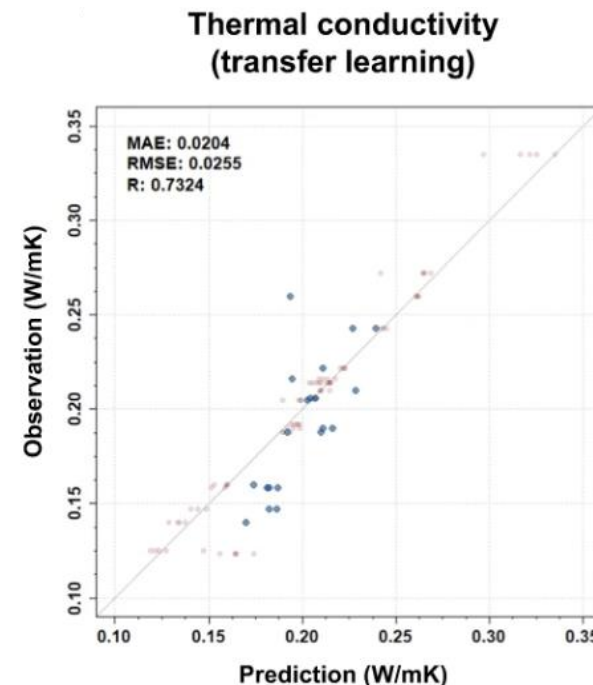
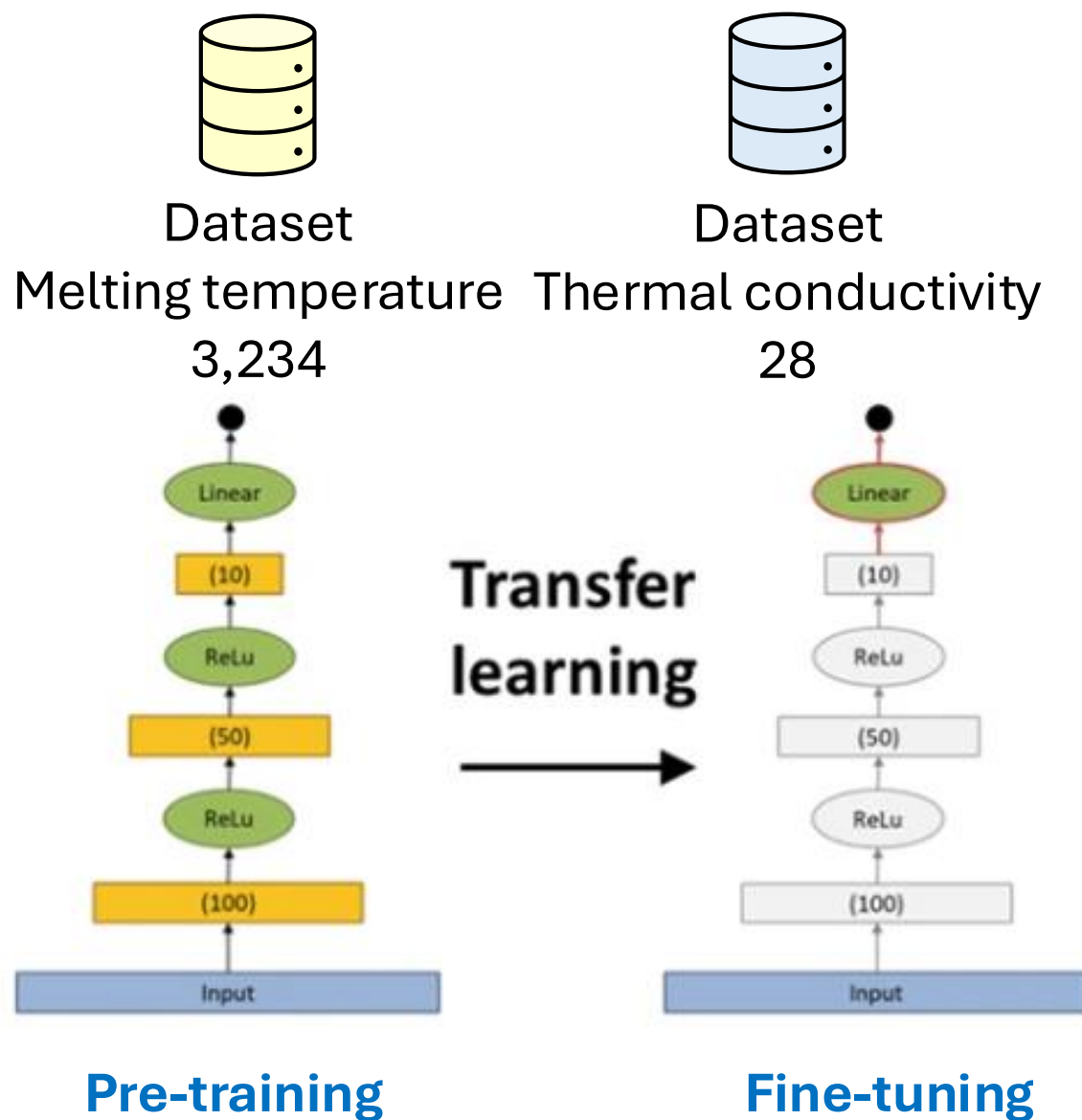
Yue, T., He, J., Tao, L. and Li, Y., 2023. High-throughput screening and prediction of high modulus of resilience polymers using explainable machine learning. *Journal of Chemical Theory and Computation*, 19(14), pp.4641-4653.

Transfer Learning

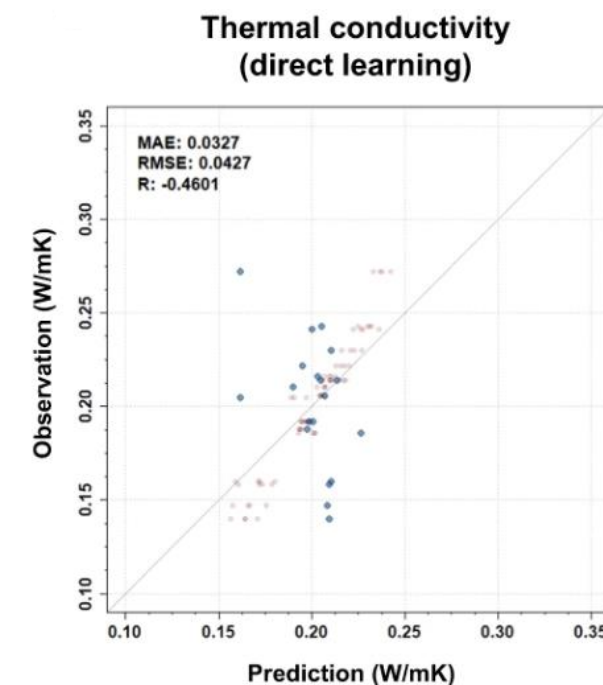


Training Prediction Models with Limited Data

Transfer Learning



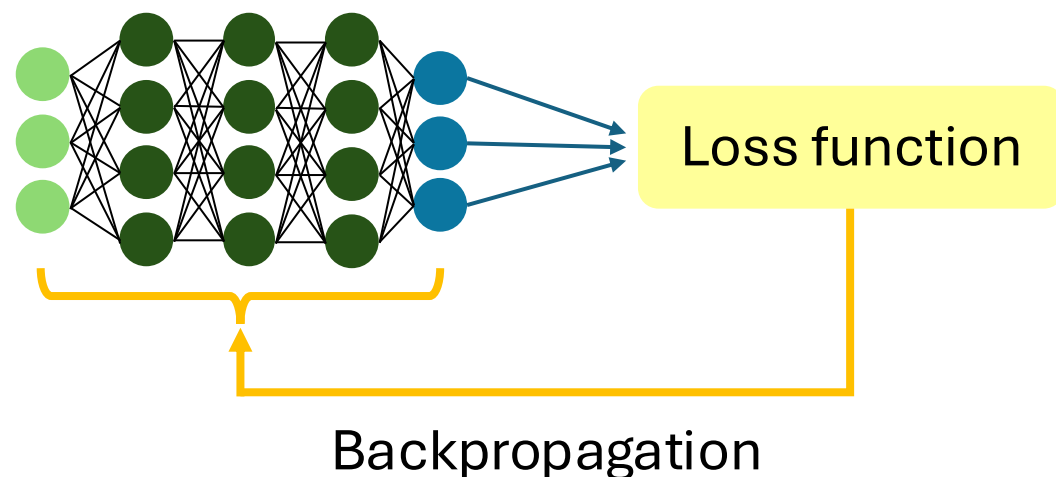
$$R^2: 0.73$$



$$R^2: -0.46$$

Wu, S., Kondo, Y., Kakimoto, M.A., Yang, B., Yamada, H., Kuwajima, I., Lambard, G., Hongo, K., Xu, Y., Shiomi, J. and Schick, C., 2019. Machine-learning-assisted discovery of polymers with high thermal conductivity using a molecular design algorithm. *Npj Computational Materials*, 5(1), p.66.

Physics-informed Neural Networks



Traditional NN: Data loss

$$\mathcal{L} = (A - \hat{A})^2 + (B - \hat{B})^2 + (C - \hat{C})^2$$

PINN: Data loss + Physics loss

$$\mathcal{L}_1 = (A - \hat{A})^2 + (B - \hat{B})^2 + (C - \hat{C})^2$$

$$\mathcal{L}_2 = f(\hat{A}, \hat{B}, \hat{C})$$

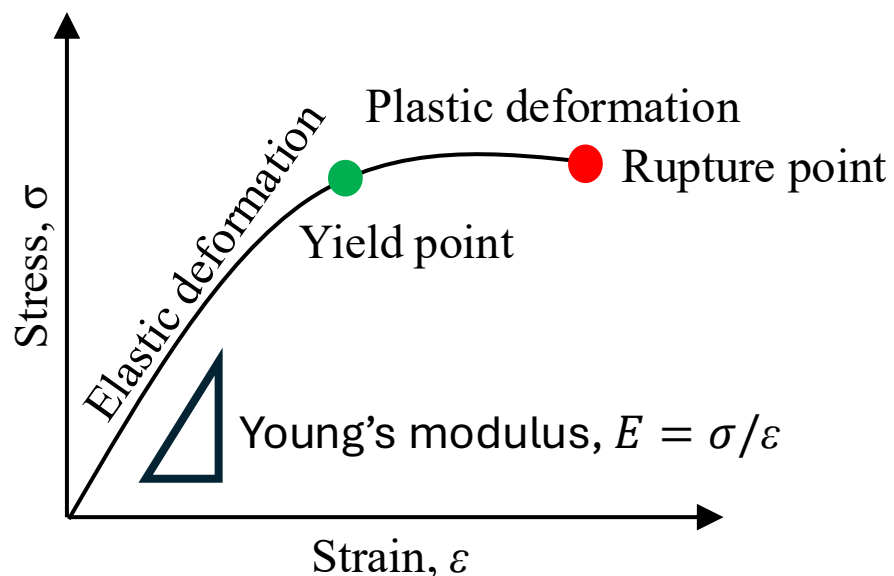
$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2$$

Only based on prediction values

A, B, C label values

$\hat{A}, \hat{B}, \hat{C}$ predicted values

$f(\cdot)$ enforce equations/ governing equations



$$\mathcal{L}_1 = (\hat{E} - E)^2 + (\hat{\sigma} - \sigma)^2 + (\hat{\varepsilon} - \varepsilon)^2$$

$$\mathcal{L}_2 = \hat{E} - \hat{\sigma} / \hat{\varepsilon}$$

$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2$$

E, σ, ε label values

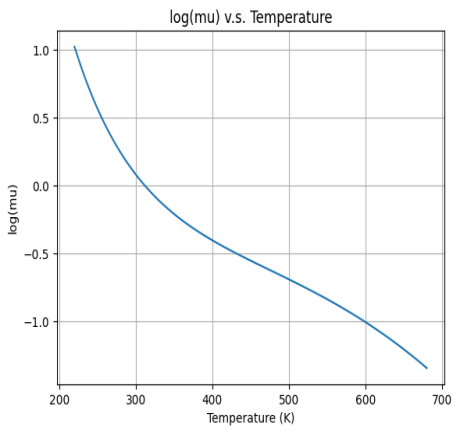
$\hat{E}, \hat{\sigma}, \hat{\varepsilon}$ predicted values

Training Prediction Models with Limited Data

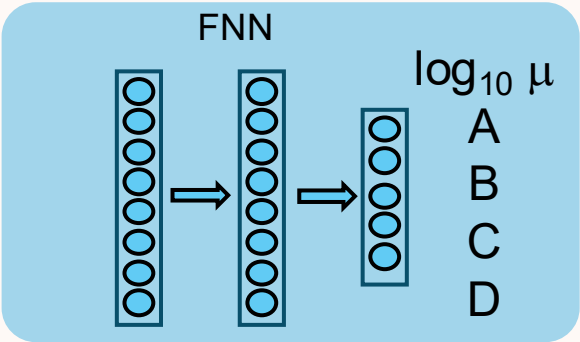
Physics-informed Neural Networks

Transport Properties of Chemicals and Hydrocarbons

$$\log_{10} \mu = A + B/T + CT + DT^2$$



6,120 unique compounds



Loss function

$$\mathcal{L}_1 = (A - \hat{A})^2 + (B - \hat{B})^2 + (C - \hat{C})^2 + (D - \hat{D})^2$$

$$\mathcal{L}_2 = [\log(\mu)' - \log(\mu)]^2$$

$$\mathcal{L}_3 = \left[\log_{10} \mu - \left(\hat{A} + \frac{\hat{B}}{T} + \hat{C}T + \hat{D}T^2 \right) \right]^2$$

$$\mathcal{L} = \mathcal{L}_1 + \mathcal{L}_2 + \lambda \mathcal{L}_3$$

Data loss

Physics loss

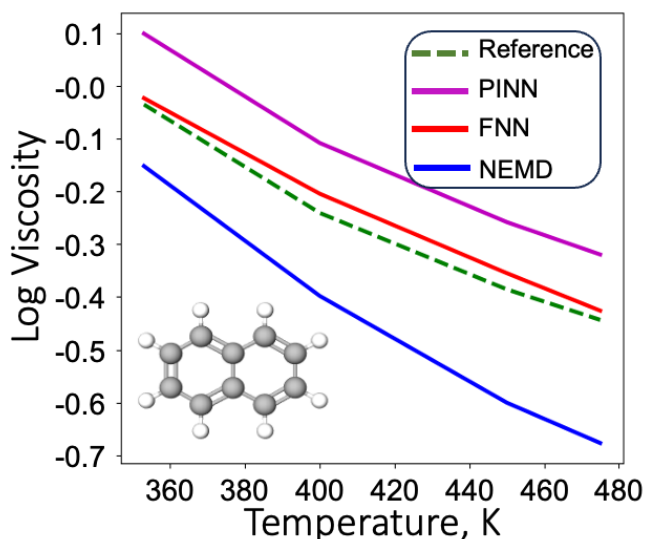
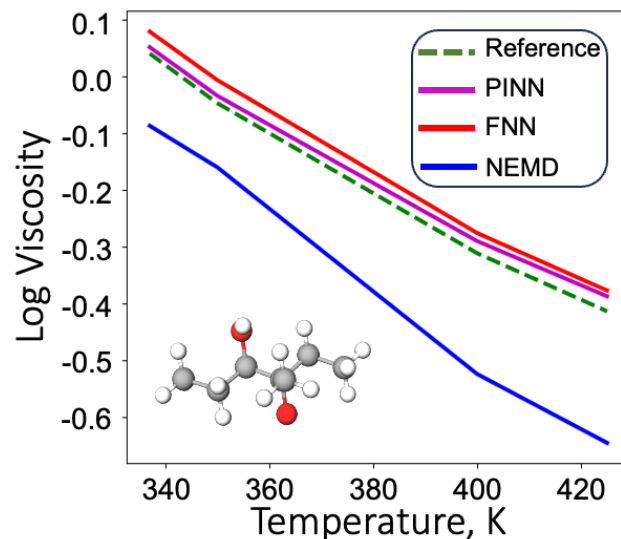
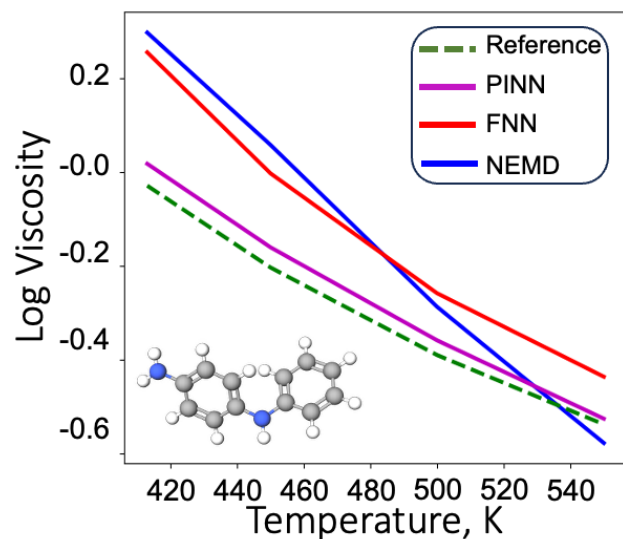
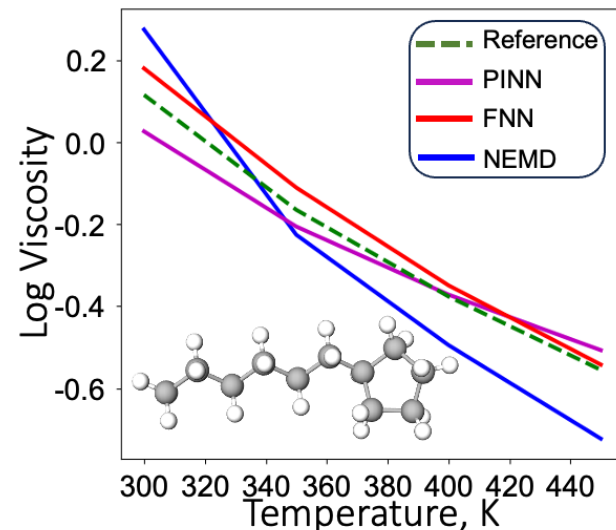
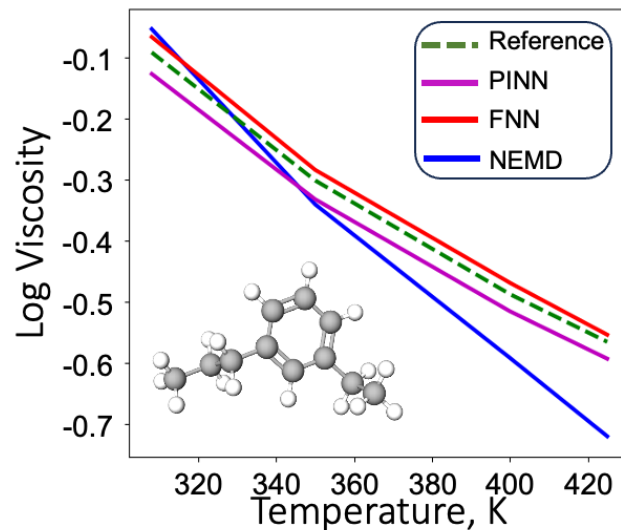
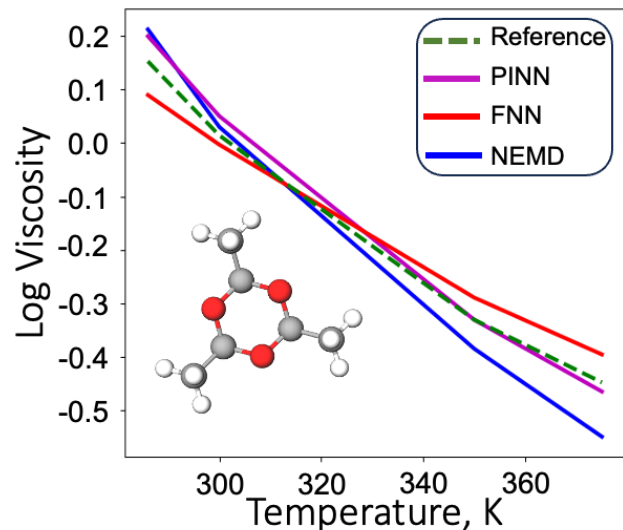
λ	R^2 (pred $\log \mu$)	R^2 (cal $\log \mu$)	R^2 (pred A)	R^2 (pred B)	R^2 (pred C)	R^2 (pred D)
0.00001	0.99	0.91	0.97	0.98	0.97	0.97
0.001	0.98	0.90	0.95	0.96	0.95	0.95
0.01	0.98	0.91	0.95	0.96	0.95	0.95
0.1	0.99	0.97	0.96	0.96	0.96	0.95
1	0.96	0.95	0.77	0.84	0.71	0.68
-	-	0.63	0.75	0.82	0.67	0.71

Only data loss

(@330k)

Training Prediction Models with Limited Data

Physics-informed Neural Networks



- **Temperature-dependency can be captured so well.**
- **Physics loss also contribute to parameter optimization.**

Question Time

Thank you!

Tianle Yue (tyue4@wisc.edu)